

Certified Stall Recovery Procedures Measured Against the Global Optimum: A Compute-Bound Dynamic Programming Approach

Nicolás D. Romero*

Business or Academic Affiliation 1, City, State, Zip Code, Country

Gabriel G. Torre[†]

Business or Academic Affiliation 2, City, State, Zip Code, Country

Roberto A. Bunge[‡]

Business or Academic Affiliation 4, City, State, Zip Code, Country

Juan I. Giribet[§]

Business or Academic Affiliation 3, City, Province, Zip Code, Country

Loss of control in flight, most often initiated by a stall, remains the leading cause of fatal accidents in general aviation. A stall recovery is a two-phase maneuver: a nose-down input to restore attached flow, then a pullout that arrests the descent. It is measured by the altitude it costs, and the certified procedures do not agree on how to fly it: one commands full power and nose-down together, the other delays power until the wing is flying again. Cast as an undiscounted infinite-horizon problem, the recovery admits a globally optimal solution by dynamic programming (DP), optimal over the whole entry envelope rather than at a nominal condition, and that guarantee is what the method is built to keep. Discretizing the state space forces the value function to be interpolated; because barycentric interpolation is non-expansive [1], the backup retains the convergence of the exact recursion and delivers the exact optimum of the discretized problem: the policies reported here carry a guarantee, not a heuristic's promise. Integrating the flight dynamics inside the GPU kernel lets that grid be fine enough to resolve the recovery. On the 3-DOF banked pullout of Bunge et al. [2] the solver resolves 31.9 million states and 91 actions in 15 minutes on a single GPU and agrees with a direct-collocation benchmark to within 1.4% of the maximum altitude lost. Applied to a realistic recovery problem, a 4-DOF symmetric stall over 14.9 million states and 451 actions with aerodynamic coefficients measured in the wind tunnel through stall and post-stall, their coupling to propulsion, and the first-order engine lag of Riley [3], the optimal policy reproduces the shape of the simultaneous-power procedure: full throttle across essentially the entire recovery envelope, an elevator switching surface pinned to the stall boundary rather than to a clock, and a pull that holds the angle of attack within a degree of that boundary. Across 1000 initial conditions of the stalled-entry set the ordering optimum, then simultaneous power, then unstall-gated power, then power-delayed holds at every entry, consistent with the flight-test ranking of Gratton et al. [4], and the exact solution decomposes the margin: most of what either certified sequence concedes is the power ramp rather than the doctrinal difference between them. Across $\pm 15\%$ weight and the achievable CG range the nominal policy continues to recover, and re-solving it for the perturbed aircraft buys 0.1 to 0.4 m on recoveries costing 9 to 46 m: nearly all of the degradation is removed by re-scaling one pre-flight scalar, the stall-speed normalization. The costliest errors are the pilot's, not the procedure's: hesitating one second before the pull costs 27 to 35 m, while the vigor of the pull is immaterial provided it is flown on the angle of attack.

*Ph.D. Candidate, Departamento de Ingeniería.

[†]Ph.D. Candidate, Departamento de Ingeniería.

[‡]Professor, Departamento de Ingeniería.

[§]Professor, Departamento de Ingeniería.

Nomenclature

ρ	air density
b	wing span
c	chord length
S	wing surface area
p_x, p_y, p_z	northward, eastward and down position
h	altitude, from the ground
u, v, w	body-x, y and z velocity
V	airspeed
α	angle of attack
β	sideslip angle
ϕ, θ, ψ	roll, pitch and yaw angles
γ	flight path angle
μ	bank angle
$\epsilon_0, \epsilon_1, \epsilon_2, \epsilon_3$	Euler parameters
p, q, r	roll, pitch and yaw rate
δ_e	elevator deflection, positive trailing edge down
δ_r	rudder deflection, positive trailing edge to the left
δ_a	aileron deflection, positive is trailing edge down of right aileron
δ_t	throttle position
$\hat{p}$	dimensionless roll rate, $\hat{p} = \frac{pb}{2V}$
L, D, Y	aerodynamic lift, drag and side force
M_x, M_y, M_z	aerodynamic rolling, pitching and yawing moment about the c.g.
C_L, C_D, C_Y	aerodynamic lift, drag and side force coefficient
C_l, C_m, C_n	aerodynamic rolling, pitching and yawing moment coefficient about the c.g.
f	system dynamic equation of motion
$V(\mathbf{x})$	value function
g	stage reward, non-positive during descent
a	vector of actions

I. Introduction

Loss of control in flight remains the largest single category of fatal aviation accidents [5], and its most common precursor is the aerodynamic stall: the wing driven past the angle of attack at which the flow can no longer stay attached, so that lift falls, drag rises, and the aircraft descends. Recovery is a two-phase maneuver, a nose-down input to restore attached flow followed by a pullout that arrests the descent, and its cost is measured in altitude: at pattern height, tens of meters separate a recovery from an accident.

For this maneuver the certified guidance is not mutually consistent. The U.S. Federal Aviation Administration (FAA) prescribes a sequential recovery, pitch nose-down first and power restored once the wing is unstalled [6], whereas the United Kingdom Civil Aviation Authority (CAA) commands full power *simultaneously* with the nose-down input [7]. Throughout this paper the two doctrines are named after their authorities, the *CAA sequence* and the *FAA sequence*, labels that name the power-timing skeletons as modeled here (the element on which the two publications genuinely differ) rather than the complete published procedures. A third tradition, deliberately delaying power until the pullout is complete (hereafter the *power-delayed* recovery), survives in training practice and was flown as a control arm in the only systematic empirical comparison to date, the fourteen-type flight-test campaign of Gratton et al. [4], which favored the simultaneous sequence (roughly two thirds of the height loss of the sequential one, which in turn cost about 90% of the deliberately delayed recovery). Flight test, however, can rank candidate procedures; it cannot say how far any of them stands from the optimum, because the optimum has never been available.

A parallel and equally unexamined gap exists in recovery *design*. Minimum-altitude-loss pullout and stall recovery are routinely posed as optimal control problems on reduced-order models of three or four states [2], under the implicit assumption that the omitted dynamics (pitch rate, sideslip, roll and yaw coupling) do not materially alter the optimal policy. This assumption is plausible but untested, because testing it requires the optimal policy of a higher-order model as a reference and the same computational obstacle has kept that reference out of reach. Making it computable is a direct consequence of the method presented here; the comparison itself belongs with the lateral dynamics and is deferred to the companion paper.

The obstacle is computational. Exact dynamic programming over a discretized state space scales exponentially with state dimension [8, 9]; for an eight-state spin model, even a coarse grid exceeds 10^{10} nodes [2], far beyond the memory available to tabular solvers that precompute and store transition tables. Practitioners have therefore relied on three alternatives, each with a structural limitation. Flight test and spin-tunnel experiments sample the envelope sparsely and at considerable cost and risk. Simplified analytic models trade fidelity for tractability [10, 11] and cannot represent the strongly coupled, post-stall regime in which spins develop. Deep reinforcement learning scales to high state dimension [12] but returns an approximate policy with no optimality certificate; its control boundaries are statistically blurred and cannot serve as ground truth against which other policies are measured.

This work removes the obstacle. We compute exact global optimal recovery policies by recasting the discretized dynamic program in a compute-bound form: rather than storing precomputed transition tables, the system dynamics are integrated on the fly within GPU registers, reducing the memory footprint from $O(N_s N_a 2^D)$ to $O(N_s)$ and bringing grids of up to $\sim 10^8$ states within reach of a single GPU. The interpolation that discretization forces is barycentric, hence non-expansive in the supremum norm, so the discretized Bellman backup retains the convergence of the exact one and the computed policy is the exact optimum of the discretized process [1, 13]. This formulation is summarized in Section II; it is the instrument of the present study rather than its subject.

This paper makes two contributions. *First*, the method: the compute-bound GPU formulation of exact dynamic programming, validated on the 3-DOF banked pullout of Bunge et al. [2] both qualitatively, reproducing the published value function and switching surfaces, and quantitatively, agreeing with a direct-collocation solution of the same optimal control problem to within 1.4% of the maximum altitude lost while resolving 31.9 million states in 15 minutes on a single GPU. *Second*, the first exact optimal stall-recovery policy computed on propulsion-coupled wind-tunnel aerodynamics: a 4-DOF symmetric stall model of a light general-aviation aircraft built on the tables of Riley [3], including his first-order engine lag with the time constant identified from his published throttle transients. The stall is *symmetric* by design, wings level and slip-free: the regime in which the certified procedures differ on power timing, and the one the flight-test comparison of Gratton et al. addressed. From this exact solution the simultaneous full-power sequence taught by the CAA *emerges* as the optimum for this model, the altitude cost of the delayed-power alternatives is

quantified across the stalled-entry set, and the pull is exposed as a sliding-mode arc riding the stall boundary. The policy remains effective across the aircraft’s achievable loading envelope, and re-solving it for the perturbed aircraft buys 0.1 to 0.4 m on recoveries costing 9 to 46 m: nearly all of the degradation is removed by re-scaling a single pre-flight scalar.

The question this framework is ultimately sized for lies one model beyond. For the developed spin, the PARE technique (power to idle, ailerons neutral, rudder opposite, elevator forward) is taught uniformly and embedded in flight manuals and type-certification guidance [6, 14], yet it was distilled from flight test, spin-tunnel campaigns, and accumulated pilot experience [15] rather than measured against a verified optimum, and posing that question requires an eight-state model. It is deferred to a companion paper; the present study establishes the method and answers the power-timing question one rung below.

The remainder of the paper is organized as follows. Section II presents the compute-bound GPU formulation, and Section III formalizes the minimum-altitude-loss recovery as an infinite-horizon optimal control problem with an absorbing set at level flight. Two case studies follow, each presenting its dynamic model and its results: Case I validates the solver on the 3-DOF banked pullout of Bunge et al. [2], and Case II scales the framework to the 4-DOF symmetric stall recovery with the propulsion-coupled aerodynamics of Riley [3], comparing the exact policy against the certified procedures flight-tested by Gratton et al. [4] and quantifying its robustness to the mass and CG variations of day-to-day operation.

II. Computational Method

A. The Memory Bottleneck of Tabular Dynamic Programming

Grid-based dynamic programming discretizes the continuous state and action spaces and iterates a Bellman backup over the resulting finite Markov decision process [8, 9]. Standard implementations precompute the dynamics once: for every state–action pair, the integrated next state is stored as the 2^D vertex indices and interpolation weights of the grid cell that contains it, so that the iterative sweeps reduce to table lookups. The stored tables occupy $O(N_s N_a 2^D)$ memory, where N_s is the number of states, N_a the number of actions, and D the state dimension. High-performance grid-based toolboxes accelerate the sweeps [16], but the tabular arrays they must hold in memory remain the binding constraint. The number is concrete: for the refined Case I grid of Sec. IV.E (31.9 million states, 91 actions, $D = 3$), the transition tables alone would occupy roughly 186 GB, beyond the memory of any single GPU, while the value function and policy arrays occupy about 0.25 GB.

B. In-Kernel Integration

The present solver stores no transition tables. During every sweep, each GPU thread is mapped to one state index, integrates the flight dynamics over the control interval with a fourth-order Runge–Kutta scheme held in registers, and evaluates the interpolated value of the resulting next state directly. The memory footprint falls to $O(N_s)$, the value and policy arrays themselves, at the price of recomputing the dynamics on every sweep. Trading stored transitions for arithmetic is a standard move in GPU computing; what makes it worth stating here is that the recomputed backup is identical to the tabular one (Sec. II.C), and that the arithmetic lands where the hardware is strongest: the profiling of Table 3 shows the solve remains compute-bound at scale, with policy evaluation accounting for 99.6% of the runtime and 41 ms of device-to-host traffic in a 15-minute run.

One implementation detail is essential for throughput. Locating the grid cell that contains the next state, and weighting its vertices, is naturally written with conditional logic, which makes threads within a CUDA warp diverge. The kernel instead uses a branchless formulation. Let x'_j be the j -th coordinate of the integrated next state and

$$n_j = \frac{x'_j - x_j^{\min}}{x_j^{\max} - x_j^{\min}} (N_j - 1) \quad (1)$$

its fractional grid index, where N_j is the number of nodes of dimension j . For each vertex $\mathbf{v} \in \{0, 1\}^D$ of the enclosing

cell, the barycentric weight is

$$W_{\mathbf{v}} = \prod_{j=1}^D [v_j \cdot \delta_j + (1 - v_j)(1 - \delta_j)], \quad \delta_j = n_j - \lfloor n_j \rfloor, \quad (2)$$

so that all threads in a warp execute the same instruction stream regardless of where their states land. In what follows, $\mathcal{V}(s')$ denotes the set of the 2^D vertices of the cell enclosing s' , so the interpolated value of a next state reads $\sum_{\mathbf{v} \in \mathcal{V}(s')} W_{\mathbf{v}} V(\mathbf{v})$.

C. Interpolation Preserves the Convergence Guarantee

Discretizing the state space forces the value function to be evaluated between grid nodes, and the choice of interpolator determines whether the discretized backup keeps the convergence properties of the exact one. The scheme above is the barycentric framework of Munos and Moore [1]: because the weights of Eq. (2) are non-negative and sum to one, the interpolation operator $\mathcal{I}$ is *non-expansive* in the supremum norm,

$$\|\mathcal{I}V - \mathcal{I}W\|_{\infty} \leq \|V - W\|_{\infty},$$

so the interpolated backup inherits the contraction structure of the exact operator instead of amplifying approximation error, and the discretized problem is consistent with the continuous one as the mesh is refined [1, 13].

This is the point of the architecture. Integrating the dynamics inside the kernel does not modify the mathematical operator at all: each thread computes the same next state, the same 2^D vertices, and the same convex weights that a tabular implementation would have precomputed, so the $O(N_s)$ footprint costs nothing in the mathematics. The complexity reduction and the preservation of the guarantee are therefore obtained together, and it is the combination that matters: the guarantee is what makes the computed policy a trustworthy optimum, and the reduced footprint is what lets the grid that carries the guarantee be fine enough to resolve the recovery.

Two properties of the present formulation should be stated explicitly, since they determine what the converged solution certifies. First, the recursion is *undiscounted* (unit discount factor): the objective is the total altitude change to absorption, and no discount factor is introduced, so the operator is non-expansive rather than a strict contraction. Convergence rests instead on the stochastic-shortest-path structure of the problem [9]: the stage reward is non-positive throughout the descent, the level-flight set $\{\gamma \geq 0\}$ and the departure set are absorbing and cost-free thereafter, and a policy that never reaches either accumulates unbounded loss and is therefore excluded by the maximization. Second, the object that converges is the optimal value function of the *discretized, interpolated, finite-action* Markov decision process induced by the grid, the action set and the control interval. Optimality is exact for that process; its distance to the continuous optimum is a discretization question, addressed for Case I in Sec. IV.E by refinement.

D. The Control Interval Δt

The integration interval Δt appearing in Algorithm 1 is a first-class solver parameter rather than a fixed constant of the method, because its value couples directly to the spatial grid through the barycentric backup. If Δt is chosen so small that the RK4 displacement covers only a small fraction of a grid cell, the interpolation weights of Eq. (2) concentrate on the origin vertex: the state effectively transitions to itself, value information propagates only a fraction of a cell per sweep, and near-ties between actions are amplified in the arg max. If Δt is too large, the one-step transition skips over cells, degrading the consistency of the multilinear backup and the detection of terminal-set crossings. The natural scale is a Courant–Friedrichs–Lewy (CFL) type condition [17]: the displacement per backup should be commensurate with one cell, $\Delta t^* \approx (\sum_i |\dot{x}_i|/h_i)^{-1}$, where h_i is the grid spacing of dimension i .

The kernel accordingly decouples the two roles the interval plays. The *control* interval, over which the action is held and the Bellman backup is applied, is a configurable parameter, selected per case study against the CFL-like scale above and reported with each discretization; internally it is subdivided into RK4 *micro-steps*, so that integration accuracy and terminal-event detection (success and crash crossings are checked at micro-step resolution) are independent of the backup interval. The solver additionally supports a state-endogenous variant that evaluates

$\Delta t(s) = \min\{\Delta t_{max}, (\sum_i |\dot{x}_i(s)|/h_i)^{-1}\}$ on the fly inside the kernel, at no memory cost, since the derivatives are already computed for the RK4 stage; the case studies reported here use the fixed-interval configuration.

E. Policy Iteration

Where prior work in this domain used Value Iteration [2], this solver implements Policy Iteration, summarized in Algorithm 1. Policy Iteration requires a full policy-evaluation step but typically converges in far fewer iterations, moving between vertices of the policy polytope rather than approaching the fixed point linearly [9]. The evaluation step, traditionally the bottleneck of the method, is here an iterative sweep over the entire state array in a single kernel launch per pass, so its cost is the per-sweep cost already profiled in Table 3.

Algorithm 1 Massively Parallel Policy Iteration for High-Fidelity Grids

```

1: Initialize:  $V(s) \leftarrow 0$  and  $\pi(s) \leftarrow \text{random}$  for all  $s \in \mathcal{S}$ , the set of the  $N_s$  grid states
2:  $policy\_stable \leftarrow \text{false}$ 
3: while not  $policy\_stable$  do
4:   {Step 1: Parallel Policy Evaluation}
5:   repeat
6:      $\Delta \leftarrow 0$ 
7:     for all thread  $s \in \mathcal{S}$  in parallel do
8:        $V_{old} \leftarrow V(s)$ 
9:        $s' \leftarrow \text{RK4}(s, \pi(s), \Delta t)$ 
10:       $V(s) \leftarrow g(s, \pi(s)) + \sum_{\mathbf{v} \in \mathcal{V}(s')} W_{\mathbf{v}} V(\mathbf{v})$  { $\mathcal{V}(s')$ : vertices of the cell containing  $s'$ ; weights from Eq. (2)}
11:       $\Delta \leftarrow \max(\Delta, |V_{old} - V(s)|)$ 
12:     end for
13:   until  $\Delta < \text{tolerance}$ 
14:   {Step 2: Parallel Policy Improvement}
15:    $policy\_stable \leftarrow \text{true}$ 
16:   for all thread  $s \in \mathcal{S}$  in parallel do
17:      $old\_action \leftarrow \pi(s)$ 
18:      $\pi(s) \leftarrow \arg \max_{a \in A} [g(s, a) + \sum_{\mathbf{v} \in \mathcal{V}(s')} W_{\mathbf{v}} V(\mathbf{v})]$ , with  $s' = \text{RK4}(s, a, \Delta t)$ 
19:     if  $old\_action \neq \pi(s)$  then
20:        $policy\_stable \leftarrow \text{false}$ 
21:     end if
22:   end for
23: end while

```

III. Problem Formulation

The minimum-altitude-loss pullout is naturally posed as a free-final-time optimal control problem with a terminal constraint, Eq. (3): find the control history that steers the aircraft from the upset state x_0 to level flight, $\gamma(t_f) = 0$, while minimizing the altitude lost along the way. Problems of this form can be solved by indirect methods based on the calculus of variations, or by direct methods such as collocation-based trajectory optimization. Either way, the solution is an open-loop control history valid for a single initial condition: recovering from a different upset requires solving the problem anew.

$$h^*(x_0) = \min_{a(\cdot), t_f} \int_0^{t_f} -V \sin \gamma \, dt \quad \text{s.t.} \quad \dot{x} = f(x, a), \quad x(0) = x_0 \text{ and } \gamma(t_f) = 0 \quad (3)$$

Two properties of Eq. (3) permit a stronger result. The running cost $-V \sin \gamma$ depends only on the state, and it vanishes on the terminal manifold $\gamma = 0$. Consequently, if the level-flight set $\{\gamma \geq 0\}$ is made absorbing (freezing the

dynamics once the flight-path angle reaches zero), the trajectory accumulates no further cost after recovery, and the problem becomes the infinite-horizon formulation of Eq. (4). Since $\gamma(t)$ is continuous, the absorbing set is entered exactly at $\gamma = 0$, where the running cost vanishes, so the integral remains finite.

$$h^*(x_0) = \min_{a(\cdot)} \int_0^\infty -V \sin \gamma \, dt \quad \text{s.t.} \quad \dot{x} = \begin{cases} f(x, a), & \gamma < 0 \\ 0, & \gamma \geq 0 \end{cases} \quad (4)$$

$$x(0) = x_0$$

The value function of Eq. (4) satisfies a stationary Bellman equation, which dynamic programming solves for all initial conditions simultaneously: the solution is no longer a single trajectory but the optimal feedback policy $a^* = \pi^*(x)$, yielding the minimum-altitude-loss recovery from every state in the flight envelope. The solver implements this minimization as the equivalent maximization of a non-positive stage reward: $g(s, a)$ in Algorithm 1 accumulates the (negative) altitude change over one control interval, the negative of the running cost of Eq. (4), so the arg max of the backup and the minimization of Eq. (4) select the same actions.

IV. Case I: 3-DOF Banked Pullout Validation against Established Literature

A. Reduced-Order Equations of Motion

Following Bunge et al. [2], the banked pullout is modeled as a three-state point mass under four simplifying assumptions: the lift coefficient and the bank rate are tracked by high-bandwidth inner loops (elevator and aileron, respectively) whose dynamics are neglected; sideslip remains zero; drag depends on the state only through the lift coefficient; and power is at idle. The state vector is $\mathbf{x} = [\gamma, V/V_s, \mu]^\top$ (flight path angle, normalized airspeed, and bank angle), and the control vector collects the two commanded quantities, $\mathbf{a} = [C_L^{cmd}, \dot{\mu}_{cmd}]^\top$:

$$\dot{\gamma} = \frac{\rho S}{2m} V C_L^{cmd} \cos \mu - \frac{g \cos \gamma}{V}, \quad (5)$$

$$\dot{V} = -g \sin \gamma - \frac{\rho S}{2m} V^2 C_D(C_L^{cmd}), \quad (6)$$

$$\dot{\mu} = \dot{\mu}_{cmd}. \quad (7)$$

The drag coefficient is a static function of the commanded lift, obtained by inverting the linear lift curve $\alpha = (C_L - C_{L_0})/C_{L_\alpha}$ and evaluating the quadratic polar $C_D = C_{D_0} + C_{D_\alpha} \alpha + C_{D_{\alpha^2}} \alpha^2$ ($C_{L_0} = 0.41$, $C_{L_\alpha} = 4.70 \text{ rad}^{-1}$, $C_{D_0} = 0.0525$, $C_{D_\alpha} = 0.2068$, $C_{D_{\alpha^2}} = 1.8712$). The aircraft is the same Grumman American AA-1 Yankee of Case II, at the flight weight used by [2] ($m = 697.18 \text{ kg}$, $S = 9.1147 \text{ m}^2$).

The control bounds encode the physics that the reduced model does not resolve. The commanded lift is limited to $C_L^{cmd} \in [-0.5, 1.0]$, a margin of 0.2 below the positive stall value ($C_L = 1.2$) and above the negative (inverted) stall value ($C_L \approx -0.7$), so that commanded-lift trajectories never enter the separated-flow regime the model cannot represent. The bank rate is limited to $|\dot{\mu}_{cmd}| \leq 30 \text{ deg/s}$, the steady-state roll rate with full aileron at stall speed [2]. This commanded- C_L , commanded-bank-rate abstraction is precisely the perfect-inner-loop assumption that Case II removes by resolving the pitch dynamics explicitly.

B. Baseline Discretization Benchmarking

To ensure a rigorous validation against established aerospace benchmarks, the state-control space was initially discretized following the rectangular grid configuration defined by Bunge et al. [2]. The flight dynamics are integrated with a fixed time step of $\Delta t = 0.1$ seconds. The stage reward $g(s, a)$ is the altitude change over the interval, non-positive during the descent, minus a quadratic penalty that ensures the smoothness of the bank rate command policy, preventing jagged control surfaces: the smoothing device introduced by Bunge et al. [2], here with a lighter weight (0.001 vs. their 0.01) that preserves the regularizing role while keeping the altitude-loss term strictly dominant:

$$g(s, a) = V \sin \gamma \Delta t - 0.001 \dot{\mu}_{cmd}^2 \quad (8)$$

The baseline grid boundaries and increments used for initial validation are detailed in Table 1.

Table 1 Discretization of state-control space (following Bunge et al. [2])

Variable	Lower Bound	Increment	Upper Bound	Units
V	0.9	0.1	4.0	$1/V_s$
γ	-180	5	0	deg
μ	-20	5	200	deg
C_L^{cmd}	-0.5	0.25	1.0	-
$\dot{\mu}_{cmd}$	-30	5	30	deg/s

On this baseline grid the full Policy Iteration solve is essentially instantaneous: Table 2 breaks down the wall-clock cost as measured with CUDA events on an NVIDIA GeForce RTX 4090. The profile is the compute-bound architecture of Section II made visible. Policy evaluation (the phase that traditional implementations amortize with precomputed transition tables) accounts for 98.5% of the runtime and executes entirely on the GPU at 12 ms per sweep; policy improvement, which evaluates all 91 actions per state, costs 0.16 ms per iteration; and the total device-to-host traffic is 0.2 ms, confirming that memory movement plays no role in the runtime. Solving the benchmark problem of [2] to full convergence takes 0.38 s end to end.

Table 2 Computational cost of Policy Iteration on 53,280 states, 91 actions (NVIDIA GeForce RTX 4090).

Phase	Total (ms)	Per Iteration (ms)
State grid construction (CPU)	0.48	–
Policy Evaluation (GPU)	375.48	12.11
Policy Improvement (GPU)	4.95	0.16
GPU → CPU Transfer	0.20	–
Total	381.11	–

Converged in 31 iterations. Timing via CUDA events.

C. Baseline Validation and Optimal Value Function Topology

Upon convergence of the Policy Iteration algorithm using the baseline discretization (53,280 states, matching the parameters established by Bunge et al. [2]), the resulting optimal value function $V^*(s)$ inherently maps the state space to the minimal altitude loss achievable from any given initial condition. Because the stage reward $g(s, a)$ is defined primarily by the incremental altitude change, the total altitude loss Δh_{min} required to return the aircraft to level flight ($\gamma = 0^\circ$) can be extracted directly from the converged grid:

$$\Delta h_{min}(V_0, \gamma_0, \mu_0) \approx -V^*(V_0, \gamma_0, \mu_0) \quad (9)$$

where the approximation accounts for the marginal numeric contribution of the bank-rate regularization penalty (8) ($0.001 \dot{\mu}_{cmd}^2$) accumulated throughout the maneuver. To validate the physical and numerical accuracy of the compute-bound framework, the converged value function was queried across slices of constant initial normalized airspeed ($V_0/V_s \in \{1.2, 2.0, 3.0, 4.0\}$). Figure 1 illustrates the resulting altitude loss contours as a function of the initial bank angle (μ_0) and the initial flight path angle (γ_0).

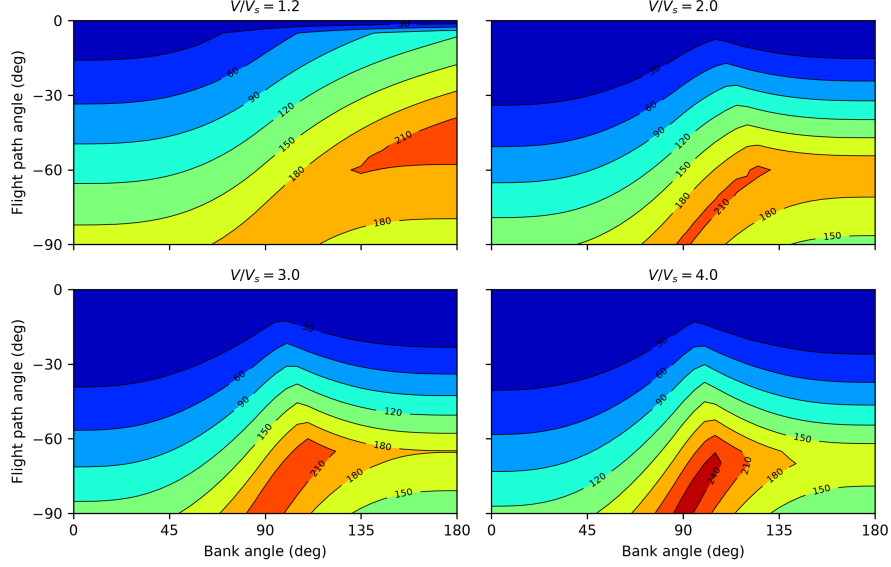


Fig. 1 Contour plots of minimal altitude loss ($\Delta h_{min} \approx -V^*$) during aircraft pullout maneuvers, evaluated on the baseline 53,280-state grid. Each subplot presents the altitude loss across the bank angle (μ_0) and flight path angle (γ_0) envelope for a fixed initial airspeed (V_0/V_s). The global topology of the solution perfectly aligns with established literature [2], validating the accuracy of the on-the-fly RK4 integration and the consistency of the barycentric interpolator.

The macro-level topology of the solution confirms the physical accuracy of the procedural GPU solver. Furthermore, the topological structure of both the value function and the optimal policy exhibits a pronounced asymmetry around the knife-edge flight condition ($\mu = 90^\circ$). This geometric skew is a direct mathematical consequence of the unequal constraints imposed on the lift coefficient. Because the aircraft’s aerodynamic threshold for positive stall is significantly higher in magnitude than its negative (inverted) counterpart, the optimal solver inherently biases the recovery roll commands toward upright attitudes. This maximizes the available normal acceleration and mathematically breaks the symmetry of the control envelope [2].

D. Optimal Control Policy: Switching Surfaces and Asymmetry

The optimal policy $\pi^*(s)$ extracted from the converged value function explicitly demonstrates a *bang-bang* control architecture. This behavior is characteristic of shortest-path optimal control problems, where applying maximum allowable control authority at all times drives the system to the terminal state (level flight) with minimal accumulated cost [2].

Figure 2 presents the optimal commanded lift coefficient (C_L^*) and optimal bank rate ($\dot{\mu}_{cmd}^*$) across the flight path and bank angle envelope, evaluated at two distinct normalized airspeeds ($V_0/V_s = 1.2$ and $V_0/V_s = 4.0$). Throughout the entire state space, the optimal actions saturate at their respective constraints: $C_L^* \in \{-0.5, 1.0\}$ and $\dot{\mu}_{cmd}^* \in \{-30, 30\}$ deg/s.

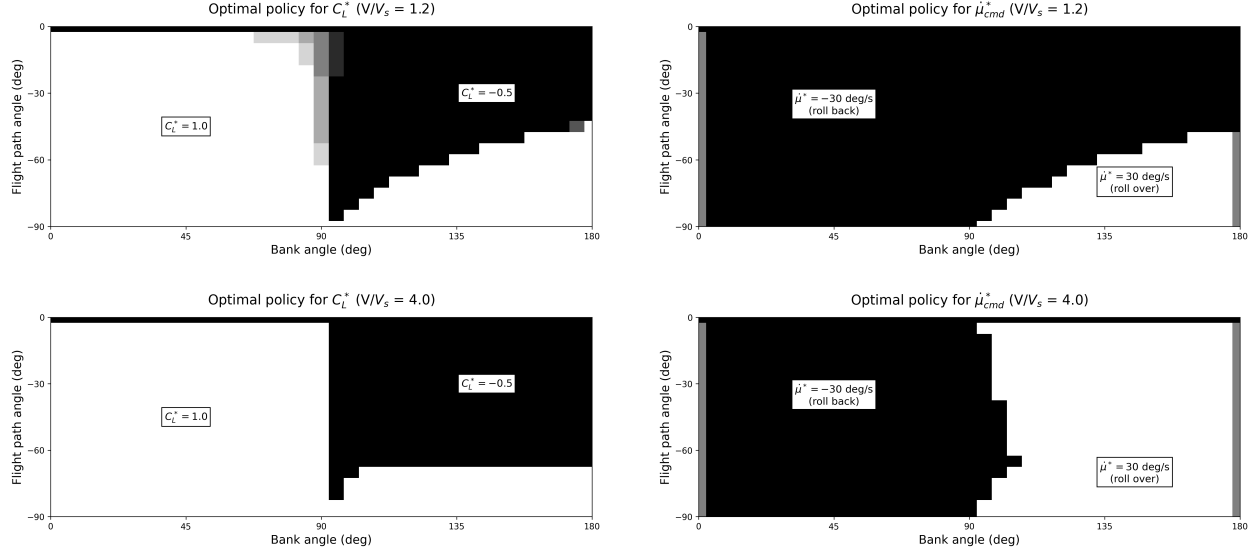


Fig. 2 Optimal policy for commanded lift coefficient (C_L^* , left column) and bank rate (μ_{cmd}^* , right column) as a function of initial bank and flight path angle. Top row corresponds to $V/V_s = 1.2$; bottom row to $V/V_s = 4.0$. The sharp boundaries between the black and white regions denote the control switching surfaces.

The boundary delineating the transition from one control extreme to the other constitutes the *switching surface*. A critical observation from the high-fidelity GPU policy is the pronounced asymmetry of these surfaces. While intuitively the switching boundary for bank rate might be expected to lie at the 90° knife-edge attitude, the optimal boundary is heavily shifted toward the inverted regime ($\mu > 90^\circ$).

This geometric skew is a direct mathematical consequence of the unequal constraints imposed on the lift coefficient (1.0 for positive angles of attack vs. -0.5 for negative angles) [2]. Because the aircraft’s aerodynamic capacity to pull up is twice its capacity to push down, the optimal solver inherently biases the recovery toward upright attitudes. As seen in the right column of Figure 2, the controller predominantly commands a -30 deg/s roll rate (rolling back to upright) even when the aircraft is banked past 90° . Rolling over to an inverted pullout ($+30$ deg/s and $C_L^* = -0.5$) only becomes optimal when the aircraft is in a deep, highly banked dive where the temporal cost of rolling back to upright outweighs the aerodynamic penalty of an inverted pullout.

Furthermore, the kinetic energy of the aircraft dynamically morphs these switching surfaces. Comparing the $V/V_s = 1.2$ scenario to the $V/V_s = 4.0$ scenario, the region where an inverted pullout is deemed optimal shrinks significantly at higher speeds. At elevated velocities, the aircraft has sufficient kinetic energy to sustain the tightest possible turning radius, making the transition back to an upright $C_L = 1.0$ recovery mathematically superior across a broader range of the envelope.

E. Scaling the Grid: The Practical Reach of the Compute-Bound Architecture

The baseline grid replicates the published benchmark at interactive speed, but its 53,280 states are coarse. To probe the practical reach of the architecture, the same problem was re-solved on a grid refined in every state dimension to 31,948,500 states (a 600-fold increase) with the same 91 actions. Table 3 reports the cost on the same GPU: full Policy Iteration converges in 67 iterations and 14.9 minutes. Two features of the profile deserve note. First, the solve remains strictly compute-bound at scale: policy evaluation accounts for 99.6% of the runtime, and the total device-to-host traffic is 41 ms of a 15-minute run. Second, the per-sweep cost grows close to linearly with the state count (12.1 ms to 13.3 s per evaluation sweep, a factor of 1100 for 600 $\times$ the states; the residual 1.8 $\times$ is occupancy and memory-traffic overhead at the larger grid, not an extra order of complexity), consistent with the $O(N_s)$ footprint of Section II.

Table 3 Computational cost of Policy Iteration on 31,948,500 states, 91 actions (NVIDIA GeForce RTX 4090).

Phase	Total (ms)	Per Iteration (ms)
State grid construction (CPU)	32.25	—
Policy Evaluation (GPU)	891257.49	13302.35
Policy Improvement (GPU)	3232.20	48.24
GPU $\rightarrow$ CPU Transfer	41.33	—
Total	894563.27	—

Converged in 67 iterations. Timing via CUDA events.

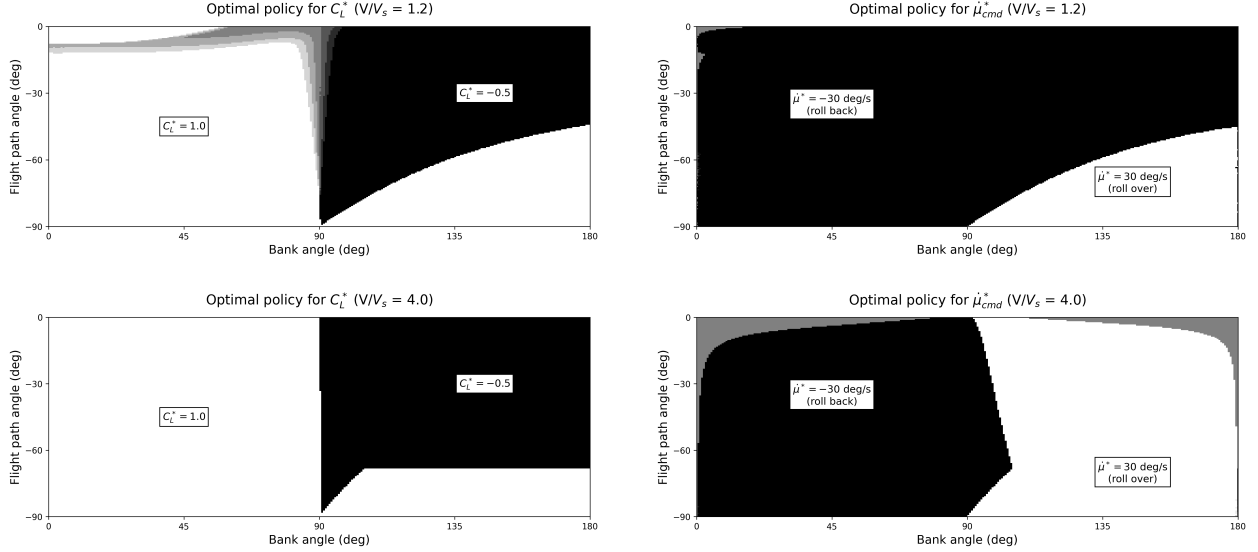


Fig. 3 The optimal policies of Fig. 2 recomputed on the 31.9-million-state grid: commanded lift coefficient (left column) and bank rate (right column) at $V/V_s = 1.2$ (top row) and $V/V_s = 4.0$ (bottom row). The switching surfaces sharpen into smooth curves and the discretization aliasing of the coarse grid is eliminated, while the global bang-bang structure is unchanged.

The refined policies (Fig. 3) preserve the global bang-bang structure of the baseline while the switching surfaces sharpen from stair-stepped bands into smooth curves. The curse of dimensionality is not defeated (work and memory still scale exponentially with the state dimension), but the constant in front of that exponential is now small enough that grids three orders of magnitude beyond the published state of the art [2] fit on a single workstation GPU. That margin is precisely what the next section spends: not on a denser grid for the same reduced model, but on a higher-fidelity one: a fourth state for the pitch dynamics, a second control for the throttle, and wind-tunnel aerodynamic tables in place of analytic polars.

F. Trajectory-Level Validation Against Direct Optimal Control

The value-function and policy comparisons above validate the solver against the published DP results of [2]; a second, independent check validates it against continuous-time optimal control. For each initial condition of Figures 3 and 4 of [2] ($V_0/V_s = 1.2$, $\gamma_0 \in \{-30^\circ, -60^\circ\}$, and initial bank angles $\mu_0 \in \{30, 60, 90, 120, 150\}^\circ$), the free-final-time problem of Eq. (3) was solved directly as a continuous-time nonlinear program (direct collocation [18], implemented in CasADi [19] with the interior-point solver IPOPT [20]), warm-started with the corresponding DP rollout. The warm start matters: the problem is nonconvex and a local NLP solver needs an initialization in the correct homotopy class (which basin of the solution space to descend), and this is precisely the global information the DP policy supplies. The two methods are therefore complementary, and their agreement is a bidirectional check: the NLP confirms the local

optimality of the DP trajectory, and the DP certifies that the NLP converged to the *global* basin.

The qualitative structure published by [2] is reproduced exactly: at $\gamma_0 = -30^\circ$ the altitude loss grows roughly threefold with initial bank (from 49 m at $\mu_0 = 30^\circ$ to 156 m at $\mu_0 = 150^\circ$), and at the near-inverted entry the optimum performs an *inverted* pullout (negative commanded lift) rather than rolling upright first; at $\gamma_0 = -60^\circ$ the near-inverted trajectory instead rolls through the vertical before pulling to an upright recovery.

Table 4 reports the comparison quantitatively, on the refined grid of Sec. IV.E. Over the nine entries the policy recovers, the two methods differ by at most 1.4% of the deepest altitude loss of each sweep and by 0.9% on average, the largest absolute disagreement being 2.1 m. Referred instead to each entry’s own loss the spread is 0.7–3.6%, and the maximum falls on the shallowest entry of the shallowest sweep, where the same ≈ 1 m disagreement is divided by the smallest denominator in the set; that is why the comparison is normalized by the deepest loss. The continuous solution is the better of the two at every entry, as the discretized policy requires: a policy holding each control over a finite interval cannot outperform one free to switch continuously, so a reversed sign anywhere would indicate that the NLP had missed its own optimum rather than that the DP had beaten it.

What remains is the rollout’s step size rather than any difference in what the two methods compute. Matching the integration — holding each policy action for the control interval but integrating it at $dt = 10^{-3}$ s — closes seven of the nine entries to within 7 cm. The tenth entry, the near-inverted $\gamma_0 = -60^\circ$, $\mu_0 = 150^\circ$ case, is excluded: the policy does not return it to level flight within the horizon.

Table 4 Trajectory-level validation of the DP policy against direct continuous-time optimal control, at the initial conditions published by Bunge et al. [2] ($V_0/V_s = 1.2$). Closed-loop DP rollouts are compared with CasADi/IPOPT solutions of Eq. (3) warm-started from the DP trajectory. Both arms terminate at the level-flight recovery $\gamma = 0$ — the NLP by its terminal constraint, the DP because the level-flight set is absorbing — so the two endpoints are directly comparable. The error e is normalized by the deepest loss of each sweep. The near-inverted $\gamma_0 = -60^\circ$, $\mu_0 = 150^\circ$ entry is excluded: the policy does not return it to level flight within the horizon.

γ_0 (deg)	μ_0 (deg)	DP (m)	NLP (m)	gap (m)	e (%)
−30	150	−157.38	−156.34	−1.04	0.67
−30	120	−126.33	−125.31	−1.02	0.65
−30	90	−90.00	−88.33	−1.67	1.07
−30	60	−62.28	−60.14	−2.14	1.37
−30	30	−50.40	−49.44	−0.96	0.61
−60	120	−194.89	−193.14	−1.75	0.91
−60	90	−152.31	−150.61	−1.70	0.88
−60	60	−121.14	−119.48	−1.66	0.86
−60	30	−109.42	−107.76	−1.66	0.86

V. Case II: 4-DOF Symmetric Stall Recovery with Riley (1985) Aerodynamics

The 3-DOF banked pullout validated the compute-bound solver against established literature, but it inherits the central simplification of the reduced-order formulation: lift coefficient and bank rate are treated as directly commanded inputs, assuming perfect, infinitely fast inner-loop tracking. To remove this assumption, the framework is scaled to a four-state longitudinal model that explicitly resolves the pitch dynamics and the post-stall aerodynamics of the airframe. The state vector is $\mathbf{x} = [\gamma, V/V_s, \alpha, q]^\top$ (flight path angle, normalized airspeed, angle of attack, and pitch rate) and the control vector is $\mathbf{u} = [\delta_e, \delta_t]^\top$ (elevator deflection and throttle position). The physical parameters correspond to the Grumman American AA-1 Yankee as tabulated by Riley [3] and are collected in Table 9 of Appendix A, together with the derived reference quantities used throughout this section, in particular the stall speed $V_s \approx 31.6$ m/s that normalizes the airspeed state.

The model is deliberately symmetric: wings level, zero sideslip, no lateral–directional states. Real stalls frequently depart asymmetrically (a wing drop at the break is the precursor of the incipient spin), and no longitudinal model can speak to that phase. The scope of this section is therefore the symmetric stall proper: the regime in which the

certified procedures disagree on power timing, and the one the flight-test comparison of Gratton et al. [4] addressed. The asymmetric departure requires the lateral–directional states and is deferred, together with the developed spin, to the eight-state companion noted in the Introduction.

A. Equations of Motion

The longitudinal dynamics are

$$\dot{\gamma} = \frac{L}{mV} - \frac{g \cos \gamma}{V}, \quad (10)$$

$$\dot{V} = -g \sin \gamma - \frac{D}{m}, \quad (11)$$

$$\dot{\alpha} = q - \dot{\gamma}, \quad (12)$$

$$\dot{q} = \frac{M_y}{I_{yy}} = \frac{\bar{q} S c C_m}{I_{yy}}, \quad (13)$$

with $L = \bar{q} S C_L$, $D = \bar{q} S C_D$, $\bar{q} = \frac{1}{2} \rho V^2$, and the dimensionless pitch rate $\hat{q} = qc/(2V)$ entering the aerodynamic coefficients below.

Notably, Eqs. (10)–(11) contain *no explicit thrust terms* ($T \sin \alpha$, $T \cos \alpha$). This is not an approximation but a direct consequence of how Riley [3] defines the power-on aerodynamic data. The wind-tunnel measurements at the powered condition were conducted with a running propeller installed on the airframe, so the force balance measured the *net axial force* acting on the complete propulsion–airframe system. Normalized by $\bar{q} S$, the tabulated drag entry is therefore a net axial-force coefficient,

$$C_{D_o}|_{C_T=0.5} = \frac{D_{\text{aero}} - T}{\bar{q} S} = C_{D,\text{aero}} - C_T, \quad (14)$$

where the thrust coefficient is defined as

$$C_T = \frac{T}{\bar{q} S}. \quad (15)$$

The negative values of the power-on drag column at low angles of attack (Table 13: $C_{D_o} = -0.347$ at $\alpha = 0^\circ$) provide direct empirical confirmation: aerodynamic drag is intrinsically positive, so a negative tabulated value can only arise when the propeller thrust dominates, yielding a net forward force. Inserting an explicit thrust term into Eq. (11) while also using the Riley power-on tables would therefore *double-count* the propulsive contribution. Thrust enters the model exclusively through C_T , which selects the operating point of the aerodynamic tables. The throttle map is calibrated such that full throttle sustains level cruise at $V_{\text{max}} = 2V_s$, giving $T = K_t \delta_t$ with $K_t \approx 1151$ N (Table 9). Definition (15) couples the propulsive and aerodynamic states: as the aircraft decelerates during the stall, $\bar{q}$ collapses and C_T rises for a fixed throttle, shifting the interpolated coefficients toward the power-on tables.

B. Propulsion-Coupled Aerodynamic Model from Wind-Tunnel Data

The aerodynamic model replaces the constant-derivative formulations used in prior stall-recovery studies [12] with the complete tabulated aerodynamics of Riley [3] (NASA TM-86309). That report documents a simulation of the airframe, but its coefficient tables are not a fitted model: they were established from full-scale powered wind-tunnel tests of the same aircraft type in the Langley 30- by 60-Foot Tunnel, with a thrust-torque balance installed between engine shaft and propeller, and it is those measurements that are used here. The constant-derivative approach carries three structural limitations: (i) post-stall lift and drag are poorly represented by low-order polynomials; (ii) all aerodynamic derivatives are treated as constants, ignoring their pronounced variation with angle of attack in the separated-flow regime; and (iii) the effect of the propeller slipstream on the airframe aerodynamics is entirely absent.

Seven coefficient tables are extracted directly from Table III of Riley [3]: the baseline lift, axial-force, and pitching-moment coefficients (C_{L_o} , C_{D_o} , C_{m_o}), the pitch-rate derivatives ($C_{L\dot{q}}$, $C_{m\dot{q}}$), and the elevator effectiveness

derivatives ($C_{L\delta_e}$, $C_{m\delta_e}$). All share the 14-point angle-of-attack breakpoint vector

$$\alpha = [-10, -5, 0, 5, 10, 12, 14, 16, 18, 20, 25, 30, 35, 40] \text{ (deg)}, \quad (16)$$

and each is provided at two propulsive operating conditions: power-off ($C_T = 0$) and a representative power-on setting ($C_T = 0.5$). The complete transcribed tables are collected in Appendix A for reproducibility; Figure 4 plots all seven coefficients at both propulsive conditions.

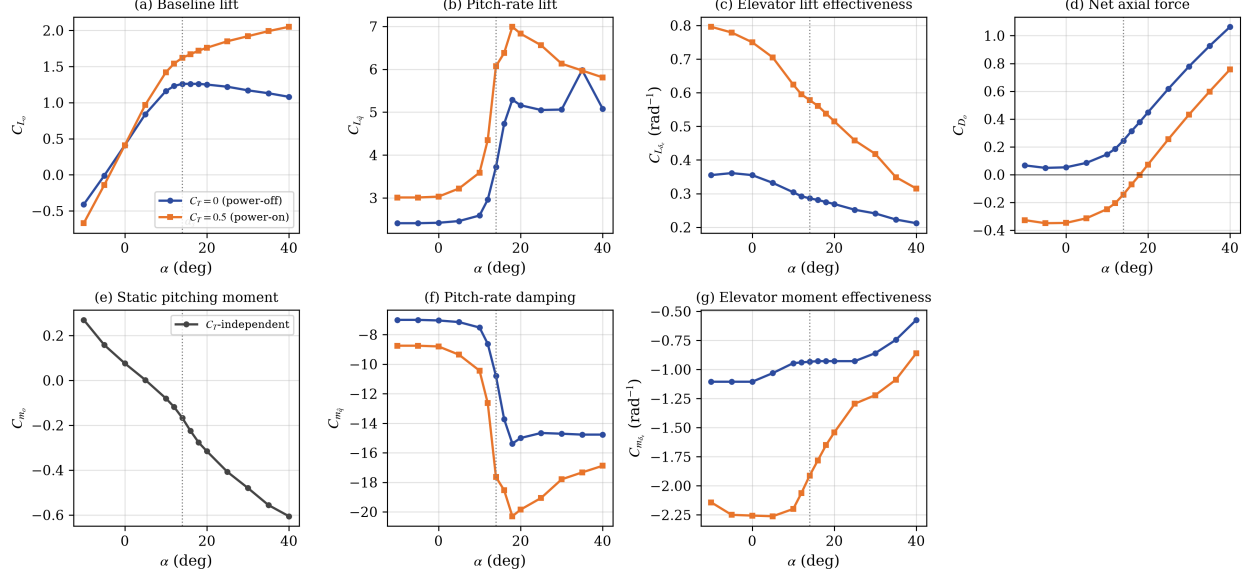


Fig. 4 The seven Riley (1985) Table III coefficients vs. angle of attack at the two tabulated propulsive conditions (one panel per table of Appendix A; C_{m_0} is C_T -independent). Three features drive the recovery physics: (a) baseline lift C_{L_o} shows the power-off flat-top stall plateau vs. monotone power-on growth from slipstream augmentation; (d) the negative power-on values of the net axial-force coefficient C_{D_o} are the embedded-thrust fingerprint of Eq. (14); (f) pitch-rate damping C_{m_q} more than doubles across the stall. The dotted line marks the stall angle $\alpha_s = 14^\circ$.

Three features of the data drive the results of this section, none of which survives a constant-derivative approximation. First, the power-off lift curve exhibits the characteristic flat-top plateau ($C_{L_o} = 1.26$ between 14° and 18°) followed by a gradual post-stall decay, while the power-on curve keeps *increasing* monotonically up to 40° (Table 10): the propeller slipstream augments the effective dynamic pressure over the wing, so powered lift remains available deep into the post-stall regime. The onset of the power-off plateau defines the stall angle used as reference throughout this work, $\alpha_s = 14^\circ$ (Table 9); it is a power-off boundary: under power the usable lift extends well beyond it, a distinction that the optimal policy will be seen to exploit. Second, the power-on axial-force column is negative below $\alpha \approx 18^\circ$ ($C_{D_o} = -0.347$ at $\alpha = 0^\circ$, Table 13): it is a net coefficient per Eq. (14), and these negative values are the empirical fingerprint of the embedded thrust discussed above. Third, the pitch-rate damping C_{m_q} more than doubles across the stall, from -7.15 at cruise to -15.4 (power-off) or -20.3 (power-on) at $\alpha = 18^\circ$ (Table 15), a stabilizing effect entirely absent from models that freeze the derivative at its cruise value.

These three features also delineate what may be fairly compared across studies. The slipstream increment is a purely aerodynamic effect: the propeller wake accelerates the flow over the wing and tail, raising the local dynamic pressure beyond what the freestream provides, and it cannot be reproduced by adding explicit thrust terms to the equations of motion. Those represent the mechanical projection of the thrust vector and leave the aerodynamic coefficients unchanged, so a model that treats thrust only as a body-axis force will systematically underpredict the normal force available during a powered recovery. Published altitude-loss figures obtained on decoupled, constant-derivative models (those of Grillo et al. [12] among them) are therefore not commensurable with the results reported here: any difference

between them conflates the solution method with the aerodynamic model. For this reason no quantitative comparison against decoupled-model results is attempted; the validation of this framework rests instead on Case I, where the benchmark and this work share the same plant.

1. Bilinear Interpolation Scheme

For any current thrust coefficient $C_T \in [0, 0.5]$ and angle of attack α , each coefficient f is evaluated via a two-dimensional interpolation: a 1-D lookup along the α axis at each of the two C_T levels, blended linearly in the C_T direction:

$$f(\alpha, C_T) = f_0(\alpha) + \frac{C_T}{0.5} [f_{0.5}(\alpha) - f_0(\alpha)], \quad (17)$$

where $f_0(\alpha)$ and $f_{0.5}(\alpha)$ are the linear table lookups at $C_T = 0$ and $C_T = 0.5$ respectively, and C_T is clipped to $[0, 0.5]$. The complete aerodynamic model is then

$$C_L = C_{L_o}(\alpha, C_T) + C_{L_{\delta_e}}(\alpha, C_T) \delta_e + C_{L_{\dot{q}}}(\alpha, C_T) \hat{q} + C_{L_{\dot{\alpha}}}(\alpha, C_T) \hat{\alpha}, \quad (18)$$

$$C_D = C_{D_o}(\alpha, C_T) + C_{D_{\delta_e}}(\alpha, C_T) \delta_e + C_{D_{\delta_e^2}}(\alpha, C_T) \delta_e^2, \quad (19)$$

$$C_m = C_{m_o}(\alpha, C_T) + C_{m_{\delta_e}}(\alpha, C_T) \delta_e + C_{m_{\dot{q}}}(\alpha, C_T) \hat{q} + C_{m_{\dot{\alpha}}}(\alpha, C_T) \hat{\alpha} + \frac{\Delta x_{cg}}{\bar{c}} C_L, \quad (20)$$

with $\hat{\alpha} = \dot{\alpha} \bar{c} / (2V)$ the dimensionless rate of change of angle of attack, formed like $\hat{q}$.

Every longitudinal coefficient tabulated by Riley is used. The elevator drag terms $C_{D_{\delta_e}}$ and $C_{D_{\delta_e^2}}$ matter because the elevator is deflected throughout a recovery, and the quadratic term gives the nose-down phase an interior optimum rather than an actuator-limited one. The rate derivatives $C_{L_{\dot{\alpha}}}$ and $C_{m_{\dot{\alpha}}}$ are reported separately from $C_{\dot{q}}$ by Riley — many models lump the two into a single damping term — and both *change sign at the stall*: $C_{m_{\dot{\alpha}}}$ goes from -0.38 at $\alpha = 12^\circ$ to $+1.80$ at 14° , so it damps below the stall and anti-damps above it. Omitting them is therefore not uniform over the domain this maneuver traverses.

Because C_L depends on $\dot{\alpha}$ while Eq. (12) makes $\dot{\alpha}$ depend on $\dot{\gamma}$, which Eq. (10) in turn obtains from C_L , the rate terms close an implicit loop. It is resolved in closed form rather than by fixed-point iteration, so that the CUDA kernel and the CPU reference evaluate the identical expression:

$$\dot{\gamma} = \frac{A C_L^- + A C_{L_{\dot{\alpha}}} k q - (g/V) \cos \gamma}{1 + A C_{L_{\dot{\alpha}}} k}, \quad k = \frac{c}{2V}, \quad A = \frac{\bar{q} S}{mV}, \quad (21)$$

where C_L^- collects the terms of Eq. (18) that do not involve $\hat{\alpha}$. The denominator stays within 1 ± 0.005 over the whole speed range of the grid. where the last term of Eq. (20) transfers the pitching moment to a center of gravity displaced Δx_{cg} aft of the Riley reference ($0.25 \bar{c}$); $\Delta x_{cg} = 0$ reproduces the nominal model and is used throughout unless stated otherwise (the robustness study of Sec. V.G exercises this term explicitly). Every derivative that was previously a scalar constant is now a function of both α and C_T , capturing the nonlinear, propulsion-coupled aerodynamics across the full post-stall envelope. All table lookups execute inside the CUDA kernel via branch-minimal linear scans over the 14 breakpoints, keeping the solver strictly compute-bound.

C. Discretization, Stage Cost, and Solver Configuration

The state-action space is discretized as detailed in Table 5, yielding $N_s = 5,648,160$ states and $N_a = 451$ actions. The angle-of-attack axis extends down to -40° so that the negative branch of the catastrophic terminal set lies *inside* the grid: its penalty value is pre-assigned to those terminal cells and propagates inward through the multilinear backup, ensuring the solver perceives the crash boundary during training rather than extrapolating past a truncated grid edge.

Table 5 Discretization of the 4-DOF state-action space for symmetric stall recovery with Riley aerodynamics.

Variable	Lower Bound	Upper Bound	Nodes	Resolution / Units
<i>State Space</i> ($N_s = 5,648,160$)				
Flight path angle (γ)	-90	5	56	~ 1.73 (deg)
Airspeed (V/V_s)	0.9	2.0	41	~ 0.028 (-)
Angle of attack (α)	-40	20	60	~ 1.02 (deg)
Pitch rate (q)	-50	50	41	2.5 (deg/s)
<i>Action Space</i> ($N_a = 451$)				
Elevator deflection (δ_e)	-25	15	41	1.0 (deg)
Throttle position (δ_t)	0.0	1.0	11	0.1 (-)

The stage reward retains the pure altitude-loss structure of the infinite-horizon formulation,

$$g(s, a) = V \sin \gamma \Delta t, \quad (22)$$

stated here in reward (maximization) form, the negated altitude term of the Case I cost, with the absorbing success set $\{\gamma \geq 0\}$. Two deliberate differences with respect to the Case I objective of Eq. (8) deserve note. First, the quadratic bank-rate smoothing penalty inherited from [2] has no counterpart here, and its absence is a consequence of the higher model fidelity rather than an omission. In the reduced-order model the bank rate is a free kinematic input ($\dot{\mu} = \dot{\mu}_{cmd}$, with no roll inertia or actuation cost), so infinitely many roll histories achieve identical altitude loss; the resulting ties fragment the discrete policy, and the quadratic penalty acts as a surrogate for the unmodeled actuation physics. In the 4-DOF formulation the actuation *is* modeled: the elevator acts through the moment chain $\delta_e \rightarrow C_m \rightarrow \dot{q} \rightarrow q \rightarrow \dot{\alpha}$, whose inertia and stall-amplified pitch damping (Fig. 4f) low-pass filter the command. Residual action ties on the constraint arc still produce bang-bang switching in the raw policy map (the discrete approximation of the singular-arc equivalent control analyzed in the sliding-mode discussion below), but the plant dynamics absorb it: the resulting $\alpha(t)$ rides the stall boundary smoothly, and the closed-loop altitude loss is insensitive to command smoothing to within 0.3 m. The objective can therefore remain pure altitude loss: the physically meaningful quantity, uncontaminated by tuning weights. Consistently, every closed-loop evaluation reported in this section executes the policy through the barycentric action blend: at an off-grid state, the commanded action is the convex combination of the argmax actions of the 16 surrounding vertices under the weights of Eq. (2). This is not a smoothing applied to the optimum for convenience; it is the solution concept the converged policy requires.

The optimal feedback law is bang-bang with a switching surface, and a differential equation closed by a discontinuous feedback has in general no classical solution on that surface: solutions are defined in the sense of Filippov [21], by the convex combination of the vector fields on either side, and the combination that holds the state on the surface is the equivalent control of sliding-mode theory [22]. Because the plant is affine in each control channel to within the bound quantified below — the elevator enters C_L and C_m linearly through $C_{L\delta_e}$ and $C_{m\delta_e}$, and the throttle through $C_T = K_t \delta_t / (\bar{q} S)$, in which the tables are interpolated linearly — a convex combination of *actions* produces the corresponding convex combination of *vector fields*. Averaging the commanded action at the switching surface therefore realizes the Filippov solution rather than approximating it, with a residual of 0.6% of the field magnitude in the elevator channel at the canonical entry ($10^{-3}\%$ in the throttle channel), measured against the mean of the constituent fields. The equivalent control recovered this way, $\bar{\delta}_e \approx -5^\circ$ over the sliding arc, is confirmed independently in Sec. V.H, where a constant elevator held open loop minimizes altitude loss at that same deflection.

Three limitations of the construction should be stated. Affinity holds per channel but not jointly, since $C_{m\delta_e}$ and $C_{L\delta_e}$ themselves vary with C_T ; blending both channels at once carries a cross-term residual, up to 18% of the field at an equal mix of the two extreme actions. Nor is the elevator channel affine in the drag axis: the quadratic term $C_{D\delta_e^2}$ of Eq. (19) makes a blended action differ from the blend of its constituent drags by the Jensen gap $\frac{1}{4} C_{D\delta_e^2} (\delta_{e,1} - \delta_{e,2})^2$. That gap scales with the *square* of the separation between the actions being mixed, and the blend mixes the argmax

of adjacent grid vertices, one degree apart: at the canonical entry it amounts to 0.01% of C_D , rising to 2.6% only if actions 20° apart were combined and to 10% at the two extremes of the elevator range. On the switching surface, where adjacent vertices disagree by one grid step, the term is numerically irrelevant; the affine argument would fail only for a blend that straddled the full control range. And the argument applies where a switching surface is actually being straddled, which is the elevator: the throttle policy commands full power almost everywhere (Sec. V.E), so intermediate throttle values returned by the blend near the absorbing boundary are an artifact of mixing with terminal cells rather than an equivalent control. Both effects are bounded by the ± 0.3 m execution band quoted throughout. Evaluated greedily instead — re-deriving $\arg \max_a Q(s, a)$ at the continuous state, which discards the Filippov combination and applies one extreme of the bang-bang at every interval — the same policy commands full throttle throughout and saturates the elevator, and the resulting chatter drives a pitch-rate overshoot costing several times the altitude. Second, the 4-DOF problem requires a catastrophic terminal set that the reduced model did not: Case I commands the lift coefficient directly within safe bounds, so its trajectories cannot depart controlled flight, whereas the 4-DOF model resolves the post-stall aerodynamics and can genuinely crash. Accordingly, the terminal set $\{|\alpha| \geq 40^\circ\} \cup \{\gamma \leq -\pi + 0.05\}$ (angle-of-attack departure beyond the range of the wind-tunnel data, or a near-vertical inverted dive) carries a fixed penalty of $-1000 V_s$, the value pre-assigned to the terminal grid cells discussed above. Each Bellman transition integrates the dynamics over a control interval of $\Delta t = 0.01$ s subdivided into ten RK4 micro-steps of 1 ms, executed entirely within GPU registers; the success and crash events are detected at micro-step resolution, so intra-interval boundary crossings are never skipped. The 4-DOF formulation requires the 16-vertex (2^4) instance of the branchless barycentric interpolation of Eq. (2).

Two further conventions govern every closed-loop number reported in this section. First, the stage reward of Eq. (22) is the altitude increment alone: all shaping weights available in the solver (pitch-rate penalty, angle-of-attack barrier, control-effort and throttle terms) are set to zero, and the only non-altitude contribution anywhere in the objective is the terminal crash penalty. This matters for the interpretation of Sec. V.D: neither full throttle nor a switch near α_s is incentivized by a weight.

Second, the altitude loss of a rollout is measured by a *dive-and-return* rule. The episode is integrated at the control interval from the stated entry; a dive is registered once $\gamma < -0.5^\circ$, and the recovery is declared complete when the flight path returns to level, either by crossing $\gamma \geq 0$ or by converging to it — γ within 0.5° of level with $|\dot{\gamma}| < 0.05^\circ/\text{s}$ sustained for 0.5 s. The second clause is needed because near $V_0 = V_s$ the optimum approaches level flight asymptotically and never crosses; genuine crossings arrive at 0.5–1.2°/s, three orders of magnitude above the tolerance, so the clause never fires on a crossing trajectory. Episodes are truncated at 15 s; truncated entries are marked wherever they are tabulated, and their altitude loss is a lower bound rather than a completed recovery.

D. Optimal Policy: Full-Power Structure and the Stall-Boundary Switching Surface

Figure 5 presents the exact DP policies for commanded elevator (δ_e^*) and throttle (δ_t^*), together with the converged value function read as minimum altitude loss, $-V(\mathbf{x})$, across the (α, γ) envelope for normalized airspeeds $V_0/V_s \in \{0.9, 1.0, 1.1\}$. Two related quantities coexist in this paper and are never interchanged: $-V(\mathbf{x})$ is the solver’s own cost-to-absorption (the Bellman recursion’s accumulated altitude reward until the trajectory enters the absorbing set), whereas every closed-loop loss reported in the tables and trajectory figures below executes the policy through the action blend of Sec. V.C under the dive-and-return stopping rule. The two fields agree in structure everywhere but differ in level by -1 – 3 m across the entry range (≈ 7.79 vs. 7.80 m at the canonical entry): the imprint of the executable smoothing and of the trajectory-level stopping rule, neither of which enters the Bellman recursion. Of the two executable readings of the converged policy, the blend is the faithful one: inside the barycentric recursion the successor state is re-averaged over the cell vertices at every step (an averaging that realizes the equivalent control of the sliding arc), and the action blend reproduces that averaging in the action space, recovering the recursion’s valuation to within 1.8 m; the literal argmax, applied to the continuous plant with no such per-step averaging, lets the bang-bang overshoot through pitch-rate inertia into losses several times larger (Sec. V.C).

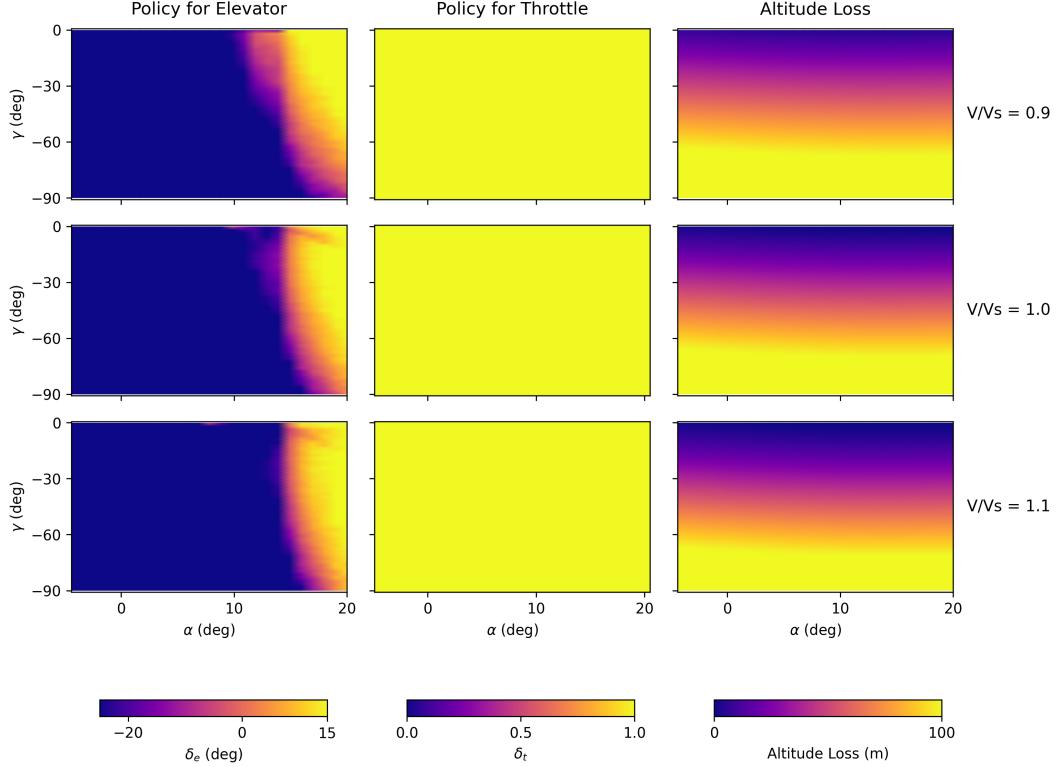


Fig. 5 Exact DP heatmaps for the 4-DOF symmetric stall recovery with the Riley (1985) propulsion-coupled aerodynamic model: optimal commanded elevator (δ_e^*), throttle (δ_t^*), and the value function read as minimum altitude loss, $-V(x)$, over the evaluated flight envelope (slice $q = 0$). $-V$ is the solver’s cost-to-absorption, not the closed-loop metric of the tables and trajectory figures, which execute the blended policy under the dive-and-return rule; the two agree in structure and differ in level by 1–3 m (see text). The elevator panels plot the *raw* arg max, so the narrow bands of saturated command that appear near $\gamma \approx 0$ should be read as the switching surface itself rather than as physical reversals: sampling the one-step Q across the full elevator range at those cells spreads it by only 0.04 m out of a 16–18 m cost-to-go (0.2%), so the extremum that wins the tie flips between adjacent rows while $-V$ stays smooth. The blend of Sec. V.C, which is what the closed loop executes, averages across those ties.

Two structural features stand out. First, the throttle policy saturates at full power essentially everywhere in the envelope of the figure. Second, the elevator policy is bang-bang with a sharp switching surface in the neighbourhood of the stall boundary: closed-loop trajectories initiated across the deep-stall envelope command full nose-down deflection ($\delta_e = +15^\circ$) until the angle of attack falls through that surface, at which point the policy switches to nose-up commands, regardless of how long that takes from each initial condition. Over the recovery-relevant columns of the converged array ($-40^\circ \leq \gamma \leq 0$, $V \leq 1.2 V_s$, $q = 0$) the switching angle has a median of 14.9° and a 5–95 percentile range of 11.9 – 15.9° : it tracks $\alpha_s = 14^\circ$, sitting about a degree above it. The surface is therefore where the optimum *stops pushing*, not where the wing has already recovered: with the elevator drag terms of Eq. (19) in the model, a nose-down deflection is not free, and the optimum stops paying for it as soon as the pitch rate it has built will carry the angle of attack through the boundary on its own. That is a distinct mechanism from the one that sets the sliding arc, which settles 2.4° below α_s (Sec. V.F): there the constraint is holding maximum lift without re-stalling, and the margin is a safety buffer against the flat-top plateau. The two are on opposite sides of α_s because they answer different questions — when to reverse the elevator during the transient, and where to ride once the descent is being arrested. The switching criterion is in any case a surface in state space, not a schedule in time. Together, simultaneous full power and nose-down elevator, pull-up once the wing unstalls, these two features are the structural fingerprint of the CAA recovery sequence.

The nature of that statement deserves emphasis, because it is not the one the remainder of this section will make. Nothing in the problem formulation refers to a recovery procedure: the stage reward is the altitude increment, augmented

only by a penalty for departing the envelope; the dynamics are wind-tunnel tables; the admissible controls are actuator limits. No candidate sequence was parameterized, scored, or searched over, and no procedure appears anywhere in Sec. V.B or in the solver configuration. The CAA structure is read directly off the converged solution of the Bellman equation: it is a property of the optimum, not the outcome of a comparison between alternatives. Everything that follows can corroborate that result or quantify what departing from it costs, but none of it can produce the result, and none of it can be undone by a different choice of scripted pilot model or a different sample of initial conditions.

The second feature is what carries the operational weight. The switch is a surface in state space pinned to α_s , not an instant on a clock, and that is precisely what makes the optimum flyable: a pilot cannot be instructed to rotate at $t^* = 0.24$ s from an entry whose onset they did not time, but can be instructed to pull as the wing unstalls. That the optimizer, free to return any measurable feedback law over a 5.6-million-state grid, returns one expressible in the same language the certified procedure already uses is the substantive finding of this subsection; the sections that follow quantify it.

E. The Optimal Recovery Is the CAA Sequence: Power Timing and Certified Procedures

As noted in the Introduction, certified guidance disagrees on the timing of power during stall recovery: the CAA sequence commands full power *simultaneously* with the nose-down pitch input [7], whereas the FAA sequence [6] prescribes the pitch reduction first, with power restored once the wing is unstalled. In the flight-test campaign of Gratton et al. [4], the CAA sequence consistently produced the least height loss, approximately two thirds of the FAA figure, which in turn was about 90% of a deliberately power-delayed recovery. The exact DP solution allows this question to be settled for the Riley model without pilot or protocol confounds. The preceding subsection has already answered it in structure; what follows makes the answer quantitative, on three independent readings of the converged solution, and then measures what the alternative sequence costs.

First, the converged policy array itself: 99.3% of the non-terminal grid states inside the recovery-relevant window ($-14^\circ \leq \alpha \leq 20^\circ$, $V \leq 1.2 V_s$, $-90^\circ \leq \gamma \leq 0$) command full throttle (92.7% over all non-terminal states of the 5.6-million-state grid), so maximum power is not a feature of one trajectory but the argmax essentially everywhere a recovery can evolve. Second, the time-domain structure: rollouts from all nine initial conditions of the grid ($\alpha_0 \in \{16, 18, 20\}^\circ \times V_0/V_s \in \{0.90, 0.95, 1.00\}$) apply full throttle from the very first control interval, simultaneously with the elevator command; in seven of the nine, a brief nose-down hold ($\delta_e = +15^\circ$) precedes the pull and is terminated not at a fixed time but at a state event: the crossing of the stall boundary $\alpha \approx 14^\circ$, reached between $t^* = 0.12$ and 0.26 s depending on the initial condition. Third, the exception that sharpens the rule: at the two lowest-energy corners ($\alpha_0 = 16\text{--}18^\circ$, $V_0 = 0.90 V_s$) the nose-down phase vanishes and the policy pulls immediately. Power-on lift grows monotonically with α well past the power-off stall (Fig. 4a), so when airspeed is scarce the optimum leans on slipstream lift rather than trading altitude for speed. The grid-wide invariant is the simultaneous full power; the duration of the pitch-down phase adapts to the energy state.

To isolate the power-timing variable, the optimal elevator schedule was left untouched and only the throttle channel was perturbed: an instantaneous application delayed by τ ; Gratton's power-delayed protocol (PD: $\tau = 2$ s followed by a 2 s ramp to full power, replicating the flight-test procedure); and a gated variant that withholds power until $\alpha < 14^\circ$: the FAA logic in idealized form, with zero recognition delay. Figures 6 and 7 report the comparison over a dense plane of stalled entries ($\gamma_0 \in [-30^\circ, 0]$, $\alpha_0 \in [14^\circ, 20^\circ]$, $q_0 = 0$, one closed-loop rollout per node on a 31×25 grid) at the three airspeeds of the policy maps of Fig. 5 ($0.90 V_s$, V_s , and the accelerated-stall slice at $1.10 V_s$), so the cost of an entry can be read against the switching surface that governs it. At the canonical IC the loss grows monotonically with any withholding of power, from 7.80 m (optimal, simultaneous) to 17.87 m for the gated FAA logic and 30.54 m for the PD protocol; the continuous delay sweep behind that ordering is taken up, as a pilot-execution error, in Sec. V.H. Table 6 reports the same three arms at every initial condition of the 3×3 grid.

Figure 6 reports the optimum over that plane. Altitude loss is governed by the flight path: at $V_0 = 0.90 V_s$ it runs from 16–18 m at level entry to ≈ 50 m from a 30° descent, a span of 35 m that widens to 41 m at V_s . The dependence on the depth of the stall is weaker by more than an order of magnitude: traversing the entire stalled range $14^\circ \leq \alpha_0 \leq 20^\circ$ moves the loss by 1–2 m, which is why the contours run nearly parallel to the α_0 axis. This is the quantitative form of the structural result of Sec. V.D: how deeply the wing is stalled sets how long the nose-down phase lasts, but the altitude

Table 6 Altitude loss Δh (m) of the optimal DP policy (CAA-like: simultaneous nose-down elevator + full power, then pull-up) vs. the same optimal pitch schedule with delayed power (FAA-like), across initial conditions ($\gamma_0 = 0$, $q_0 = 0$). PD = Graton’s power-delayed ($\tau = 2\text{ s} + 2\text{ s}$ ramp); gated = power withheld until $\alpha < 14^\circ$.

α_0	V_0/V_s	Optimal	$\tau=0.5\text{ s}$	$\tau=1\text{ s}$	PD	gated
16	0.90	-17.19	-20.53	-24.48	-39.79	-28.72
16	0.95	-6.57	-10.06	-13.93	-28.94	-15.32
16	1.00	-0.62	-1.92	-6.06	-21.16	-7.39
18	0.90	-16.37	-20.11	-24.01	-39.29	-27.08
18	0.95	-7.16	-10.80	-14.67	-29.67	-16.61
18	1.00	-0.82	-2.64	-6.67	-21.71	-8.42
20	0.90	-15.85	-20.09	-23.99	-39.24	-26.58
20	0.95	-7.80	-11.66	-15.54	-30.54	-17.87
20	1.00	-1.07	-3.50	-7.39	-22.38	-9.50

[†] No return to level flight within 15 s (shallow powered descent).

that phase costs is set by how fast the aircraft is already falling when the recovery begins.

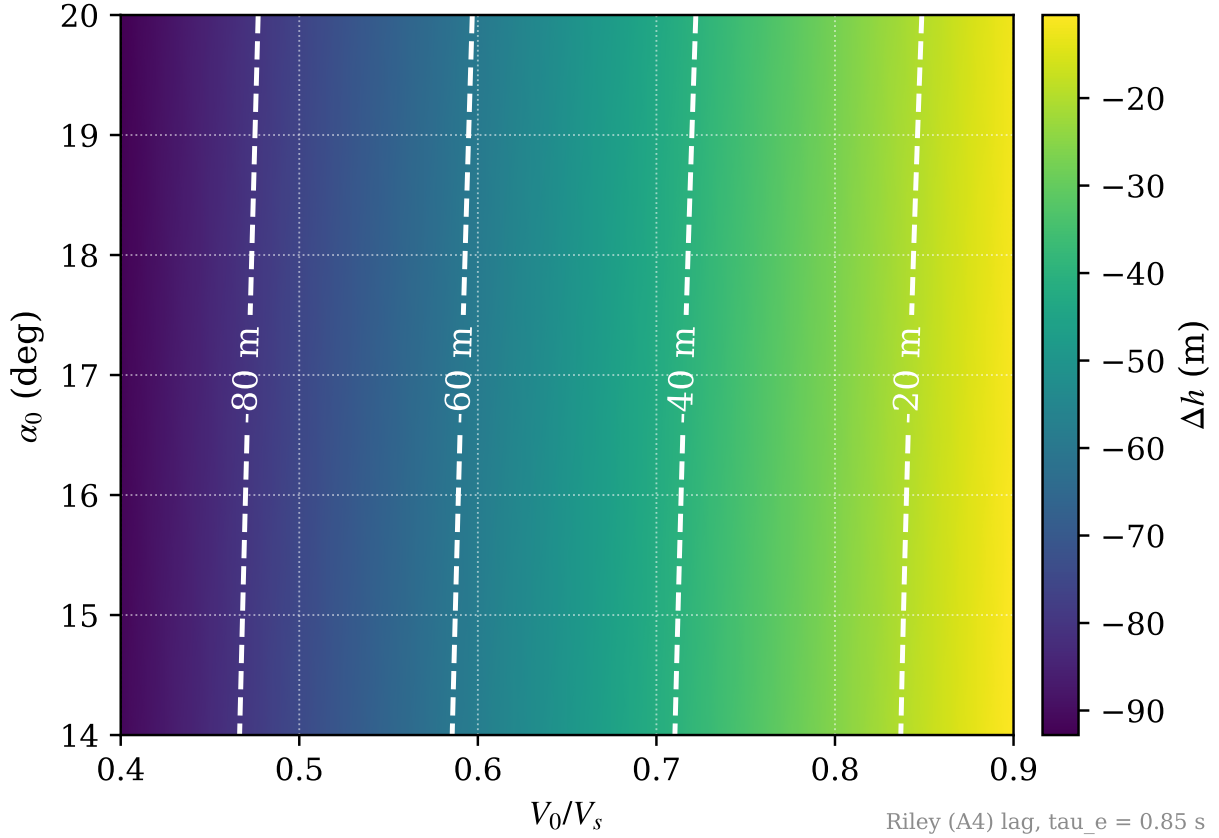


Fig. 6 Altitude loss of the exact optimum over the stalled-entry plane (α_0 abscissa, γ_0 ordinate, $q_0 = 0$), at the stalled-entry airspeeds of the policy maps of Fig. 5; one closed-loop rollout per node on a 31×25 grid, labeled contours on a shared scale. The field is governed by γ_0 (a 35 m span at $0.90 V_s$, 41 m at V_s) and only weakly by α_0 (under 2 m across the entire stalled range).

Figure 7 prices the procedures against that optimum: each panel maps $\Delta h_{\text{opt}} - \Delta h_{\text{proc}}$, the extra altitude the procedure loses from the same entry, with every arm flying the DP-optimal elevator schedule so that power timing is the only difference between the rows. All three rows repeat one shared scale anchored at zero excess, so the ranking is read

directly down each column. The CAA row is the floor imposed by the engine: reaching full power through a 2 s ramp rather than instantaneously costs 5–8 m at every stalled entry, and no procedure flown on this aircraft can avoid it. That floor is a property of the low-energy regime, not of the ramp: at $1.10 V_s$ the same ramp costs under 5 m and falls below 0.1 m at the shallowest entries, because an aircraft entering the stall with surplus airspeed does not need the power it is waiting for. The FAA row is that floor plus the price of the doctrine, and the increment has structure: gating power on the unstall costs most toward the low-speed, shallow-descent entries: at $V_0 = 0.90 V_s$ entered level the doctrine effect has a median of 3.9 m and peaks at 4.3 m (median excesses of 7.6 m for the CAA sequence against 11.1 m for the FAA), and decays to a median of 1.5 m by a 30° descent, so a single headline number understates the penalty exactly where altitude margin is scarcest. Most of what either certified sequence gives up to the optimum is nevertheless the engine, not the doctrine, a distinction that bounds how much any revision of the published procedure could recover. The power-delayed row puts both in perspective: Gratton’s deliberate 2 s pause costs 13–23 m over the optimum, dwarfing the engine floor and the doctrine effect alike.

The third airspeed of the policy maps, $1.10 V_s$, was swept identically, and over its valid rows ($\gamma_0 \leq -1^\circ$, 750 nodes) the ordering is the same at every node. Its level-entry row, however, is not a stall recovery, which is why the accelerated-stall slice is reported here rather than mapped. At $\gamma_0 = 0$ with $V_0 > V_s$ the entry sits on the boundary of the DP’s absorbing level-flight set (the formulation rightly treats such a state as already recovered), and under full power the vertical force balance lifts it into that set within the first integration step ($\dot{\gamma}(0) \approx +4^\circ/\text{s}$ over $\alpha_0 = 16\text{--}20^\circ$, the slipstream lift increment of the Riley tables of Sec. V.B raising C_L at fixed α enough to close the vertical balance above unity); past the boundary the recovery problem, and with it the policy, is simply not posed. A single degree of initial descent absorbs the transient entirely (the climb then peaks below 0°), so the boundary is sharp. The distinction mirrors the roles of the two figures: the policy maps of Fig. 5 display the feedback law over *states*, which recoveries legitimately traverse at $V > V_s$ mid-dive, whereas the entry maps measure outcomes over *stalled entries*, which are well posed on $V_0 \leq V_s$. Across the two mapped airspeeds and the valid rows of the third (2300 entries), the ordering is unbroken: $|\Delta h_{\text{opt}}| \leq |\Delta h_{\text{CAA}}| \leq |\Delta h_{\text{FAA}}| \leq |\Delta h_{\text{PD}}|$, without a single exception.

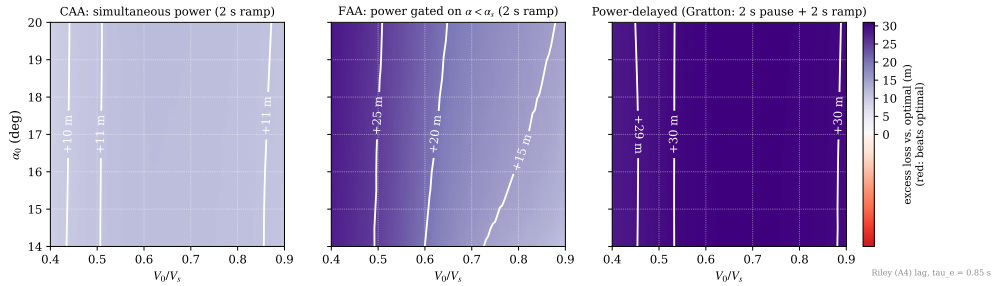


Fig. 7 Excess altitude loss of each recovery procedure over the optimum of Fig. 6, $\Delta h_{\text{opt}} - \Delta h_{\text{proc}}$, from the same entries; all arms fly the DP-optimal elevator schedule, so power timing is the only difference between rows. **Top:** CAA, full power from $t = 0$ through a 2 s ramp: the floor imposed by the engine itself. **Middle:** FAA, power gated on $\alpha < \alpha_s$, then the same ramp: the floor plus the doctrine penalty, largest toward low-speed, shallow entries. **Bottom:** Gratton’s power-delayed protocol (2 s pause, then the ramp). The three rows repeat one shared viridis scale anchored at zero excess, so the ranking reads directly down each column. On every one of the 1550 nodes shown, the ordering optimum $\leq$ CAA $\leq$ FAA $\leq$ PD holds without exception.

Both figures are confined to $q_0 = 0$ and to one plane of a four-dimensional entry set. Table 7 removes that restriction: every arm is flown from the same 1000 Latin-hypercube samples spanning all four entry states over the certified domain $V_0 \leq V_s$, so the interactions the planar maps cannot show are included. The ranking survives intact: the CAA sequence loses less than the FAA sequence at 100% of the sampled entries, by a median of 1.32 m. But the certificate also records how thin the margin becomes at its narrowest, 0.05 m, which no aggregate figure would reveal.

Table 7 Coverage of the stalled-entry set: every arm flown from the same 1000 Latin-hypercube samples of $\gamma_0 \in [-30^\circ, 0]$, $V_0/V_s \in [0.90, 1.00]$, $\alpha_0 \in [14^\circ, 20^\circ]$, $q_0 \in [-20, 20]^\circ/\text{s}$. Unlike the planar maps of Fig. 7, this samples the interactions between entry states, so the ranking is certified over the set rather than on a slice of it. Percentiles describe the uniform sampling box, not an operational entry distribution; the per-entry ranking reported below is distribution-free.

Arm	Δh (m)		excess over optimum (m)	
	median	5th pct	median	95th pct
DP optimum	-24.08	-43.61	—	—
CAA (simultaneous)	-30.67	-49.63	6.56	8.05
FAA (gated on $\alpha < \alpha_s$)	-32.05	-51.19	8.12	10.60
Power-delayed (Gratton)	-41.85	-59.42	17.71	22.68

The CAA sequence loses less than the FAA sequence at 100.0% of the sampled entries (median advantage 1.50 m, smallest 0.05 m); 998 of 1000 entries closed the recovery under every arm.

The ratios, not the absolute losses, are what may be compared against the flight tests: Gratton’s protocol decelerates at 1 kt/s to the natural stall break ($V \lesssim V_s$, α just past critical), which corresponds to the level-entry edge ($\gamma_0 = 0$) of the low-speed panel. Under matched power-ramp modeling (2 s ramp in both arms), the simultaneous sequence loses 0.61–0.62 of the power-delayed figure along that edge at $V_0 = 0.90 V_s$, in close agreement with the ≈ 0.6 implied by the flight-test chain ($\text{CAA} \approx \frac{2}{3} \times \text{FAA} \approx 0.9 \times \text{PD}$). Notably, Gratton’s fleet included the Grumman AA-5A, a direct sibling of the AA-1 modeled here, which lost 120 ft under the CAA sequence versus 230 ft under the FAA sequence.

F. Time-Domain Recovery and the Sliding-Mode Regime

The certified-procedure comparison of the preceding subsection still flies the DP-optimal elevator, which no procedural pilot reproduces. For a like-for-like comparison, both certified sequences were scripted with an identical pilot model: nose-down $\delta_e = +15^\circ$ held until the unstall event, then a closed-loop pull that holds $\alpha \approx 13^\circ$ ($\delta_e = \text{clip}(K_\alpha(\alpha - \alpha_{tgt}) + K_q q)$ with $K_\alpha = 10$, $K_q = 2$), with power ramping to full over 2 s starting at $t = 0$ (CAA) or at the unstall event (FAA). This subsection examines one of those recoveries in the time domain and tabulates the rest. Figure 8 shows the closed-loop recovery from the canonical deep-stall initial condition (level flight, $\gamma_0 = 0$; $\alpha_0 = 20^\circ$; $V_0 = 0.85 V_s$; $q_0 = 0$) under the exact DP policy, overlaid with the two scripted procedures flown from the same entry. The entry speed is $0.85 V_s$ rather than the $0.95 V_s$ used before the propeller was calibrated: under Riley’s thrust the aeroplane recovers from $0.95 V_s$ without ever leaving controlled flight, so that entry no longer poses the problem this section is about.

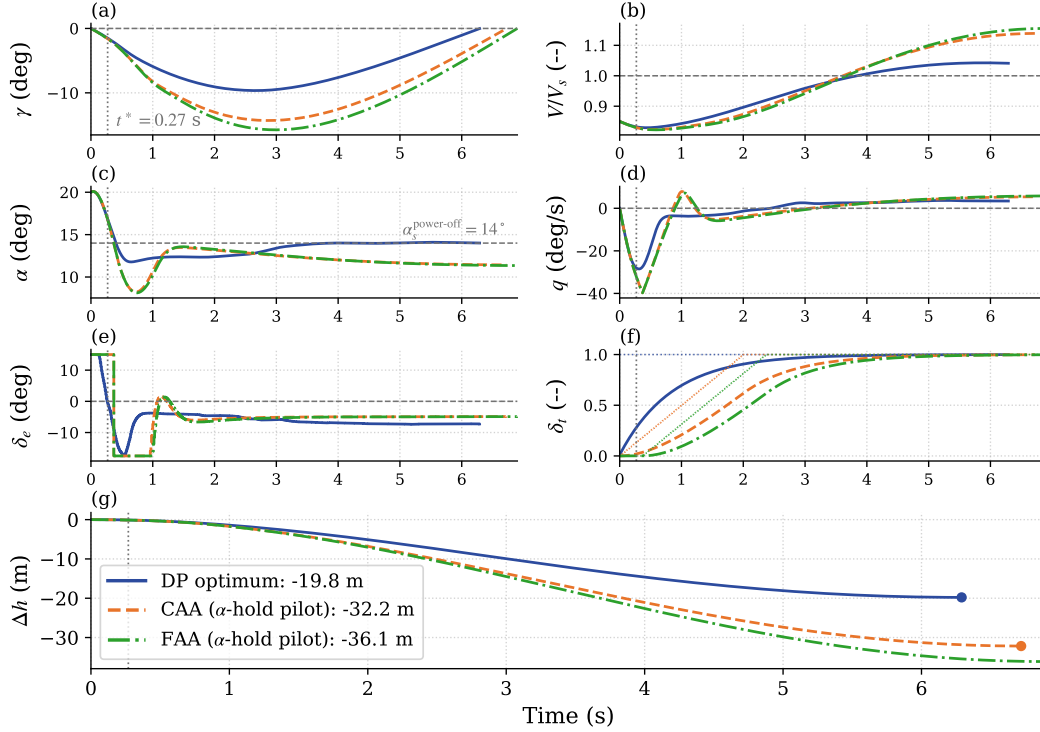


Fig. 8 Time-domain comparison at the canonical deep-stall initial condition: exact DP policy (blue, solid) vs. the scripted CAA (orange, dashed) and FAA (green, dash-dotted) procedures of Table 8. Panel (f) isolates the protocol difference, and separates what the pilot commands (dotted) from what the engine delivers (solid) through Riley’s first-order lag, $\tau_e = 0.85$ s: full power commanded at $t = 0$ (DP), a 2 s ramp from $t = 0$ (CAA), the same ramp from the unstall event (FAA); final losses in panel (g). The vertical dotted line marks the DP policy’s elevator-switch instant $t^* = 0.27$ s, the reference event for the pilot-error study of Sec. V.H. Both scripted pilots are held to the same pull authority as the optimum ($\delta_e \geq -17.50^\circ$ at this entry), so the figure isolates the timing of the power application rather than confounding it with how hard the pilot hauls back. The DP traces execute the policy through the barycentric action blend of Sec. V.C; the intermediate deflections and throttle settings of panels (e)–(f) are convex combinations of the discrete extremes, not commands the optimizer selected: the underlying policy is bang-bang, and holds full throttle throughout. The dashed line in panel (c) is the *power-off* stall boundary of the Riley tables, $\alpha_s = 14^\circ$; the superscript qualifies the boundary, not the state of the aircraft, which holds full throttle throughout. Under power the lift curve continues to rise past that value, so the optimum’s excursion above the line is not a re-stall.

The recovery unfolds in three phases. The policy first commands full nose-down elevator and full power at once; the angle of attack falls from 20° to the stall boundary in 0.41 s, while the engine, lagged by τ_e , needs 1.95 s to deliver 90% of the thrust commanded at $t = 0$. At the boundary the policy transitions to the pull-up and enters a *sliding-mode* regime: the elevator command oscillates at the control rate between nose-up and mild nose-down deflections, actively regulating the pitch rate so that the angle of attack rides at the stall boundary ($\bar{\alpha} = 13.3^\circ$, peaks of 14.1° , i.e. marginally past α_s) for the remainder of the pullout. The time-averaged elevator deflection over this phase is only -6.4° : the chattering between action extremes is the discrete action set’s approximation of the intermediate *equivalent control* [22] that holds the state on the constraint surface, the classical signature of a singular arc realized by a bang-bang policy [23]. By continuously surfing the usable lift curve without paying the post-stall drag penalty of a secondary stall, the policy maximizes the normal acceleration throughout the arc, arresting the descent with a total altitude loss of 19.8 m from the canonical initial condition. That figure is dominated by the engine: the same policy flown on an instantaneous propeller loses 9.4 m, so about half of the optimum’s own altitude loss is the cost of waiting for thrust that has been commanded but not yet produced. The optimum is not exempt from the lag; it is simply the arm that commands power earliest.

Absolute losses throughout this section inherit the uncertainty of τ_e itself, 0.8 ± 0.2 s, which is worth approximately ± 2 m here; the margins *between* arms are far less sensitive, and are what the conclusions rest on.

The two scripted procedures overlay the same panels. Their pitch-control law is identical by construction (same nose-down input, same α -hold pull, panel e), so every difference between the orange and green trajectories, and between them and the optimum, traces back to the power channel of panel (f). Three features stand out. First, both procedures dive about half again as deep as the optimum ($\gamma_{min} = -14.3^\circ$ and -15.8° vs. -9.7°): a ramp stacked on top of the engine lag cannot arrest the initial sink rate the way the optimum's immediate command does. Second, the CAA and FAA trajectories are nearly indistinguishable in every state variable for the first second (the FAA gating delay is only 0.37 s at this entry), yet that brief power deficit compounds through the dive into a 3.9 m gap at recovery (-32.2 vs. -36.1 m). Third, the α -hold pilot overshoots the unstall (α dips to 8.1 – 8.2° before settling on its 13° target, panel c) while the optimum descends only to 11.8° before settling on its 13.3° sliding floor: the procedures lose their altitude not through poor pitch flying but through later, slower power.

The remaining entries are tabulated. Table 8 reports the results, in the format of the flight-test tables of Gratton et al. [4] (methods across the columns, height loss per entry), with their aircraft-type axis replaced by the initial condition and the exact optimum added as the reference column; the parenthesized full-pull variants play the role of their controllability annotations, recording what happens when the pull technique degenerates. The CAA sequence loses less altitude at every one of the nine initial conditions, at 87–92% of the FAA figure (mean 90%), a milder advantage than Gratton's two thirds because the scripted FAA pilot restores power at the very instant of unstall, the most favorable possible reading of the FAA sequence; any recognition delay moves it toward Gratton's power-delayed protocol. Two further observations follow. The gap between the exact optimum and either scripted procedure (-19.8 vs. -32.2 – -36.1 m at the canonical IC) is the margin attributable to optimal control over procedural flying, and it is 11.4–17.4 m across the nine entries, about three times the CAA-vs-FAA gap at the same entry. And replacing the closed-loop pull by an open-loop full pull ($\delta_e = -25^\circ$ held, the one arm deliberately left at the actuator stop rather than capped at the optimum's own authority) re-stalls the wing at every entry, at $\alpha = 33$ – 35° , and none of the eighteen resulting trajectories returns to level flight at all: they are still stalled when the 15 s horizon expires, by which point 101–114 m have been lost (parenthesized entries). The recovery outcome is therefore not merely more sensitive to *how* the pull is flown than to *when* power is applied — the two are not commensurable, because one degrades a recovery and the other prevents it. This is taken up in Sec. V.H.

Table 8 Altitude loss Δh (m): exact DP optimum vs. the scripted CAA and FAA recovery procedures, simulated on the Riley model as defined by Gratton et al. (all values are simulation results of this work). Nose-down $\delta_e = +15^\circ$ until $\alpha < 14^\circ$; pull-up holds $\alpha \approx 13^\circ$ (α -hold pilot); power ramps to full over 2 s from $t = 0$ (CAA) or from the unstall event (FAA). In parentheses: the same maneuver with the pull-up instead flown open loop at full deflection ($\delta_e = -25^\circ$ held). The overdone pull re-stalls the wing at every one of the nine entries, driving α to 33 – 35° against the 14° boundary, and none of them recovers: the parenthesized figures are the altitude lost when the 15 s horizon expires with the aircraft still stalled, not a recovery cost. The canonical entry is set in bold: it is the maneuver resolved in the time domain in Fig. 8.

α_0	V_0/V_s	DP optimum	CAA	FAA
16	0.80	-26.30	-38.82 (-110.74)	-42.07 (-113.55)
16	0.85	-18.46	-30.59 (-106.28)	-33.19 (-108.66)
16	0.90	-10.95	-22.32 (-101.49)	-24.54 (-103.58)
18	0.80	-26.91	-39.57 (-110.33)	-43.57 (-113.67)
18	0.85	-19.07	-31.26 (-105.57)	-34.69 (-108.62)
18	0.90	-11.51	-22.98 (-100.79)	-25.92 (-103.48)
20	0.80	-27.63	-40.49 (-110.31)	-45.01 (-114.00)
20	0.85	-19.79	-32.17 (-105.50)	-36.12 (-108.92)
20	0.90	-12.19	-23.88 (-100.67)	-27.33 (-103.76)

For the Riley model the conclusion is therefore unambiguous: the minimum-altitude-loss recovery has the CAA structure (full power applied simultaneously with the nose-down input, followed by a pull initiated at the stall boundary), and the exact optimum, the delayed-power sweep, and the scripted procedures all rank power timing exactly as the flight tests of [4] do.

G. Robustness of the Nominal Policy to Mass and CG Variations

The policy is solved once, for the nominal aircraft; the aircraft actually flown differs from flight to flight in weight and longitudinal CG position (fuel state, payload and its placement). This subsection quantifies the altitude cost of that mismatch: the nominal policy is flown, unchanged and uninformed, on aircraft perturbed by mass changes of up to $\pm 15\%$ and CG shifts of up to $\pm 7\%$ of the chord (the moment-transfer term of Eq. (20)), over the full IC grid: 441 closed-loop rollouts. The negative mass range covers the realistic flight-to-flight envelope of the AA-1 (a light two-seater whose useful load is a large fraction of its weight); the Riley reference weight itself sits at the certificated ceiling, so the +10 to +15% rows represent an *overload* stress case beyond the certificated maximum, included deliberately, overloading being a recurrent factor in general-aviation accidents. Three modeling conventions define the operational scenario. The perturbation enters the dynamics only: the observation normalization keeps the *nominal* stall speed, so the policy believes the aircraft is nominal. Initial conditions are placed relative to the *real* stall speed of the perturbed aircraft, $V_s \sqrt{m/m_{nom}}$: the physics of the stall break belongs to the real aircraft; the mismatch lives only in the policy's state normalization. And the pitch inertia is left unperturbed, since day-to-day load sits near the CG and its inertia contribution is second order. One consequence of the second convention should be stated at the outset: because the entry speed is scaled by the real V_s while the observation is not, 84 of the 441 rollouts (19%, the light rows at the lower entry speeds) begin below the $V/V_s = 0.900$ floor of the state grid, where the barycentric read clamps to the boundary rather than extrapolating. Those cells measure the policy at a saturated query, and are discussed as such below.

Figure 9 reports the resulting mass–CG matrix at the canonical IC. Along the CG axis, at nominal weight, the loss varies smoothly from 1.9 m worse than nominal at the most forward position to 1.3 m better at the most aft, and every cell recovers: a forward CG adds nose-down moment that the pull must overcome, an aft CG relieves it. The nominal policy remains well matched across the whole envelope because the quantity that organizes it, the switching surface set by the stall boundary, is a property of the aerodynamic tables, not of the trim: shifting the CG moves the moment balance, not the $\alpha \approx \alpha_s$ boundary that triggers the pull.

The mass axis is where the mismatch itself becomes visible, and below nominal weight it does something that at first reading looks wrong: the benefit of being light does not grow with how light the aircraft is. At the nominal CG, removing 5% of the mass saves 1.2 m, removing 10% saves only 0.6 m, and removing 15% costs 0.5 m more than the nominal aircraft. The non-monotonicity is not aerodynamic. Repeating the same three cells with the observation normalization corrected — the policy told the real stall speed of the lighter aircraft, everything else unchanged — recovers a strictly monotone trend of 1.97, 3.75 and 5.34 m saved, in increments of 1.97, 1.78 and 1.59 m per 5% of mass removed. The physics is smooth, mildly sublinear and favourable throughout; what bends it back on itself is the single scalar the policy normalizes with.

The mechanism is direct, and at the light end it is not merely a degradation but a saturation. A lighter aircraft stalls slower, so at the same fraction of its *own* stall speed it is flying faster in absolute terms than the policy, dividing by the nominal V_s , believes. The policy therefore treats it as closer to the stall than it is and flies with margin it does not need. Below roughly -10% of the reference weight the mis-scaled observation leaves the state grid altogether: the canonical entry maps to $V/V_s = 0.876$ against a grid floor of 0.900, and the barycentric read clamps to the boundary rather than extrapolating, so the policy stops responding to airspeed entirely. The consequence is visible only past the metric's horizon. Integrated for 60 s with the stopping rule removed, the nominally normalized light aircraft never damps: it holds a sustained phugoid whose γ excursion grows from $[-8.6^\circ, +5.7^\circ]$ at -5% to $[-12.2^\circ, +12.6^\circ]$ at -15% , carrying V/V_s as low as 0.70 and thus below the grid floor in the low half of every cycle, so the reported loss is the first zero crossing of that oscillation. The oscillation therefore appears before the entry state itself leaves the grid: it is enough that the recovery transient does. The same policy on the same aircraft, given the correct V_s , settles to within $\gamma \in [-0.7^\circ, 2.2^\circ]$ across all three light rows and climbs away. The error grows with the mass deficit until it consumes the entire advantage of being light: at -15% the correctly normalized policy recovers in 2.5 m and the nominally normalized one in 8.3 m. Nearly six metres, on an aircraft that is easier to recover than the one the policy was designed for, attributable to one mis-scaled state. It is also the cheapest defect in the study to repair: take-off weight is known before the flight, and dividing by the right V_s costs nothing.

Read along the nominal-weight row, the CG axis grows monotonically and without surprises, from +1.86 m of excess loss at the most forward position to -1.28 m at the most aft, with no CG at which the policy degrades abruptly.

What that row deliberately does *not* report is the distance to a policy re-solved for the CG of the day. An operator has no such policy, so the gap to it measures a quantity nobody can act on, whereas the degradation of the one policy actually flown is the operationally meaningful figure.

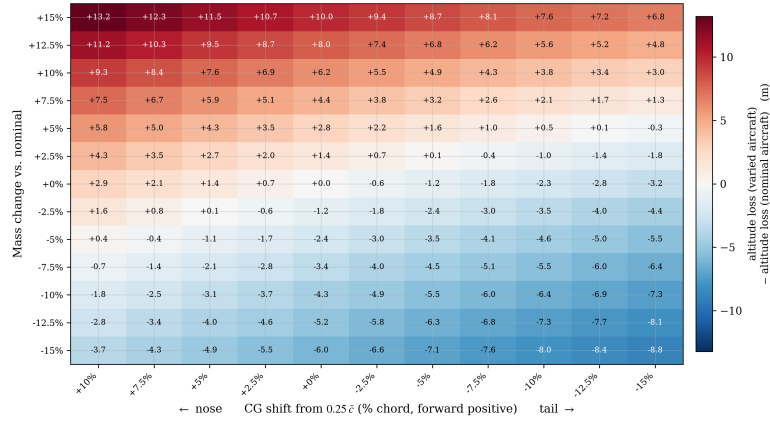


Fig. 9 Excess altitude loss (m) of the *nominal* policy flown on the perturbed aircraft, relative to the same policy on the nominal aircraft (-7.80 m, canonical IC), over the mass change ($\pm 15\%$ of the reference weight) and CG shift ($\pm 7\%$ of $\bar{c}$, aft positive) envelope. Positive (red) = additional loss; negative (blue) = less loss than nominal. Rows are read as the weight of the day, columns as its placement. At nominal weight the aft-CG credit is genuine; in the light rows the reported value is the first zero crossing of a sustained oscillation and understates the loss. Asterisks mark the +10 and +15% rows, where the dive is arrested but the aircraft settles into a *steady* shallow descent and never regains level flight: their values are losses at the 15 s cutoff, and lower bounds. Daggers mark the two cells that reach $\gamma = 0$ while still stalled; both are analysed in the text.

Two cells of the matrix carry a dagger rather than an asterisk, and they defeat the metric in a third way. At -15% weight with the CG at $+5$ and $+7\%$ of the chord, the trajectory does cross $\gamma = 0$ within the horizon, but crosses it still stalled, at $\alpha = 16.8^\circ$ and 18.7° against $\alpha_s = 14^\circ$. Both cells require the aft CG and the light mass together. The CG enters the pitch stiffness as $(\Delta x_{cg}/\bar{c}) \partial C_L / \partial \alpha$, and $\partial C_L / \partial \alpha$ is 3.35 rad^{-1} below the stall against 0.22 above it, so an aft CG barely alters the nose-down phase but removes 30% of the stiffness the moment the aircraft unstalls: α rebounds at $27^\circ/\text{s}$ instead of the $3\text{--}5^\circ/\text{s}$ of the forward-CG cells. At -15% weight that rebound reaches 19.3° , against 13.7° at nominal weight where it never re-stalls at all, and the aircraft retains lift margin enough to hold $|\gamma| < 0.7^\circ$ throughout, so no dive develops and the latch of the stopping rule trips on a -0.69° excursion. Since the terminal set of the DP is the purely geometric $\{\gamma \geq 0\}$, with no condition on α , that counts as an arrival. Their apparent $7.7\text{--}7.8$ m credit — the episode closes having lost 0.1 m, because nothing ever pointed down — measures when the metric fires, not a faster recovery, and is not the envelope's best corner.

Above nominal weight a single failure mode appears. No trajectory in the 441 rollouts crashes or departs: the angle of attack never exceeds 21.0° anywhere in the matrix (the deep-stall entry region itself, 19° clear of the $\pm 40^\circ$ departure boundary), and in the heavy rows it never exceeds its entry value: the policy that believes the aircraft light never re-stalls the aircraft that is heavy. The dive is always arrested. Extending the simulation horizon to 120 s, however, shows that the heavy rows do not merely recover slowly: they converge to a *steady* powered descent ($\gamma_{ss} = -0.44^\circ, -1.84^\circ, -3.10^\circ$, with steady sink rates of $0.25, 1.04, 1.78 \text{ m/s}$ at $+5, +10, +15\%$ mass) that never re-crosses $\gamma = 0$: integrated for 120 s with the stopping rule removed, all three record zero crossings.

The $+5\%$ row deserves a note, because the matrix scores it as recovered while it shares this failure mode exactly. Its steady descent, -0.44° , lies just inside the $\pm 0.5^\circ$ convergence band of the stopping rule (Sec. V.C), so the rule reads the settled flight path as level and closes the episode at 12.4 s. The aircraft is in the same permanent shallow descent as the two rows above it; it is merely shallow enough to pass for level flight. What separates the overload rows is therefore not the presence of the failure but its steepness.

This equilibrium is physical, and its origin can be stated exactly. Sustained level flight at speed V requires,

simultaneously, trimmed lift and a non-negative speed rate at full throttle:

$$C_L(\alpha, C_T; \delta_e^{trim}(\alpha)) \geq \frac{2mg}{\rho V^2 S}, \quad C_D^{net}(\alpha, C_T) \leq 0, \quad \alpha \leq \alpha_s, \quad (23)$$

with $\delta_e^{trim}(\alpha) = -C_{m_o}/C_{m_{\delta_e}}$ from the moment balance. The axial threshold is zero, rather than a thrust term, because the Riley tables are propulsion-coupled: thrust enters through the C_T axis and is already embedded in the tabulated coefficient, so C_D^{net} is drag *net* of the thrust it carries and is negative under power. The condition $C_D^{net} \leq 0$ therefore reads “the embedded thrust at least pays the drag”, i.e. the aircraft is not decelerating; it is not a claim that profile drag is negative. Sweeping the Riley tables at the airspeed the policy holds ($V_{ss} = 1.010, 1.021, 1.040 V_s$), the lift and axial conditions are each satisfiable, but over *disjoint* ranges of α . Taking the +15% case: below $\alpha = 10.4^\circ$ the axial condition holds comfortably — the embedded thrust exceeds the drag, C_D^{net} reaching -0.10 — but the trimmed lift falls short of the $C_L = 1.34$ that level flight demands. Lift is met only on approach to α_s , from $\alpha = 13.6^\circ$, and by then the axial condition has long reversed ($C_D^{net} = +0.09$ at $\alpha = 14^\circ$). No single trimmed α satisfies both, so no configuration of the control surfaces sustains level flight. The width of that gap is what sets the steepness of the resulting descent: it spans 10.4° – 13.6° at +15% but only 10.9° – 11.4° at +5%, which is why the lightest overload settles at -0.44° and the heaviest at -3.10° . The overweight aircraft at this speed sits on the back side of the power curve, where the lift it needs costs more net drag than the embedded thrust can pay. The force balance therefore closes on a descending path, $\sin \gamma_{ss} = -\bar{q} S C_D^{net} / (mg)$, evaluated with the full drag polar including the elevator terms at the trimmed deflection. It reproduces the simulated γ_{ss} to two decimals at all three overloads ($-0.44^\circ, -1.84^\circ, -3.10^\circ$), which confirms that the residual descent is a force-balance equilibrium and not a grid artifact. Pulling harder is not an escape: the stall boundary α_s does not move with mass, and beyond it the wing genuinely re-stalls. Level flight becomes feasible only above $V^* = 1.030, 1.065, 1.150 V_s$, that is, at $\approx 1.005, 1.015, 1.072$ of the heavy aircraft’s *own* stall speed, precisely the operating point a correctly normalized policy would hold. The deficit is modest: 30% more available thrust would make level flight feasible at the very speeds the policy already holds (V^* drops below V_{ss} for all three overloads); but the same feasibility is available at zero hardware cost by exiting the dive 1.9–10.6% faster, which the corrected normalization delivers.

Two consequences follow. First, for these cells the altitude loss of “the recovery” is not a single number: the maneuver decomposes into a transient (the dive and its arrest, essentially complete at the 15 s horizon of the matrix) and a steady mismatch descent whose cost is a *rate*. The matrix reports the former; the steady sink rates above complete the description, and the loss at any later time is their sum. For the asterisked rows the cutoff therefore conceals nothing, since the mark itself announces that the number is a lower bound; for the +5% row, scored as a recovery, the 0.25 m/s that follows is not announced by the matrix and is recorded here instead. Second, the failure is recoverable by design: the aircraft *can* complete the recovery (V^* exists, barely above its own stall speed); what is missing is the policy’s knowledge that it must exit the dive faster. That knowledge enters through a single scalar: the stall-speed normalization, computable from the pre-flight weight. Re-scaled, the policy would read $V/V_s^{real} \approx 0.97$ – 0.99 at the states where it now reads ≈ 1.0 , hold the recovery longer, and exit above V^* .

In summary, the overweight failure chain is: the policy arrests the dive and attempts the level-off; at the airspeed its nominal normalization deems sufficient, *sustaining* the required lift is aerodynamically infeasible under the Riley coefficients (the drag of producing it exceeds the embedded thrust, the back side of the power curve), so the aircraft settles into a steady shallow descent; and it never accelerates out of the deficit because the mis-scaled normalization hides that it is flying too slow for its weight. The aircraft can recover; the policy does not know it must fly faster. The failure is informational, not aerodynamic.

Two remarks bound the operational reading of the heavy rows. First, the terminal state is not a hazard state: the aircraft ends wings-level in a shallow powered descent, at full throttle, well below the stall boundary: a condition from which a pilot simply takes over, accelerates, and lands. From a deep-stall entry, the nominal policy has saved the aircraft; what it fails to close is the last few degrees of flight-path angle, not the recovery. Second, the overload rows are a synthetic stress test, not a claim about reachable flight states: the simulations *place* the aircraft at the stalled initial condition by construction, whereas an airframe loaded 15% beyond its certificated maximum might well lack the climb performance to reach such a scenario at altitude in the first place. The rows measure the policy, not the plausibility of

the flight that would precede them.

The perturbation ranges map onto the aircraft’s actual loading decisions. The Riley reference weight sits at the certificated ceiling, effectively two occupants with full tanks. Against that reference, flying with half fuel is -4% (≈ 30 kg); leaving the second seat empty is -11% (≈ 80 kg); a light pilot flying solo on half tanks reaches the -15% edge of the matrix. The 5% grid steps thus bracket the aircraft’s discrete loading items almost one to one: each row down is roughly one decision: less fuel, one occupant fewer. The rows above nominal, by the same token, correspond to no legal loading state at all. Seven percent of the chord is 8.5 cm of CG travel, which covers every CG position actually achievable by loading this airframe: the side-by-side seats and the spar-mounted fuel sit essentially at the CG, so only the baggage shelf moves it; even its full 45 kg allowance, a meter aft, shifts the CG by only $\approx 5\%$ of the chord. The matrix of Fig. 9 therefore spans, in physical terms, essentially every state this aircraft can depart in, legal or not. Across all of it the nominal policy arrests the dive without ever departing or re-stalling; what it cannot do, beyond the certificated weight, is finish the level-off, a failure that the pre-flight weight, applied to a single normalization scalar, removes.

H. Sensitivity to Pilot Execution Errors

The preceding subsections established what the optimum is and that it survives the parameter variations of a working aircraft. What remains is the pilot. A human departs from the optimal sequence along three axes, and this subsection places all three on a common scale so their magnitudes can be compared: applying power late, switching from nose-down to the pull late, and pulling with less than the optimal deflection. Every experiment perturbs the closed-loop optimal policy directly, one axis at a time, from the canonical entry.

The power axis is the one already introduced in Sec. V.E, where it served to separate the certified procedures; measured as an execution error rather than a doctrine, it is the mildest and the most regular of the three. Figure 10 sweeps it continuously: with the optimal elevator schedule held fixed and only the throttle channel delayed by τ , the loss grows monotonically and very nearly linearly, at ≈ 7.7 m per second of delay. The sweep’s zero-delay point now coincides with the reference optimum to within 0.01 m: the throttle-averaging artifact that previously separated them, an intermediate throttle returned by the blend near the absorbing boundary, is removed once the policy of the terminal cells is filled from their nearest interior neighbour rather than left at its initialisation. The rate is remarkably insensitive to where the aircraft starts: repeating the sweep at all nine initial conditions yields parallel curves, and a $\tau = 2$ s delay costs between 13.1 and 16.0 m of excess over each entry’s own optimum across the whole grid. A pilot who is half a second slow on the throttle pays 3.9 m, wherever the stall began.

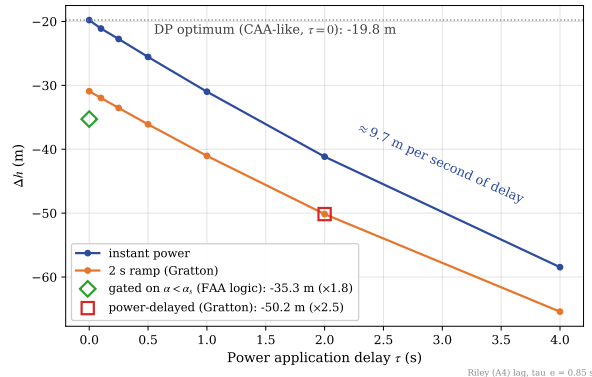


Fig. 10 The power axis: altitude loss vs. power-application delay τ at the canonical initial condition ($\alpha_0 = 20^\circ$, $V_0 = 0.95 V_s$), with the optimal elevator schedule held fixed, for the instantaneous and 2 s-ramp power models. The dotted line marks the DP optimum ($\tau = 0$, CAA-like simultaneous power); the diamond, the gated variant (idealized FAA logic); the square, Gratton’s power-delayed protocol ($\tau = 2$ s delay plus 2 s ramp). The loss grows monotonically, by ≈ 7.7 m per second of delay: the linear reference against which the two pitch axes of Fig. 11 should be read.

The two pitch axes are measured the same way. In the first, the policy is followed until its nose-down $\rightarrow$ pull-up

transition, the nose-down command is then held for τ_s additional seconds, and the policy resumes. The second axis, the pull itself, is flown in two modes that bracket how a human might execute it: *closed loop*, where the policy is followed but its pull commands are capped at a maximum deflection $|\delta_e^{pull}| \in [5^\circ, 25^\circ]$ (a pilot who limits authority but keeps regulating on the aircraft's response); and *open loop*, where after the switch a constant deflection $-\delta_e^{pull}$ is simply held, with no feedback on the angle of attack (a pilot who hauls and freezes). Both interventions leave the nose-down phase untouched: they act from the switch instant $t^* \approx 0.24$ s marked across the panels of Fig. 8: the first experiment perturbs the *timing* of that transition, the second the *execution* of the pull that follows it.

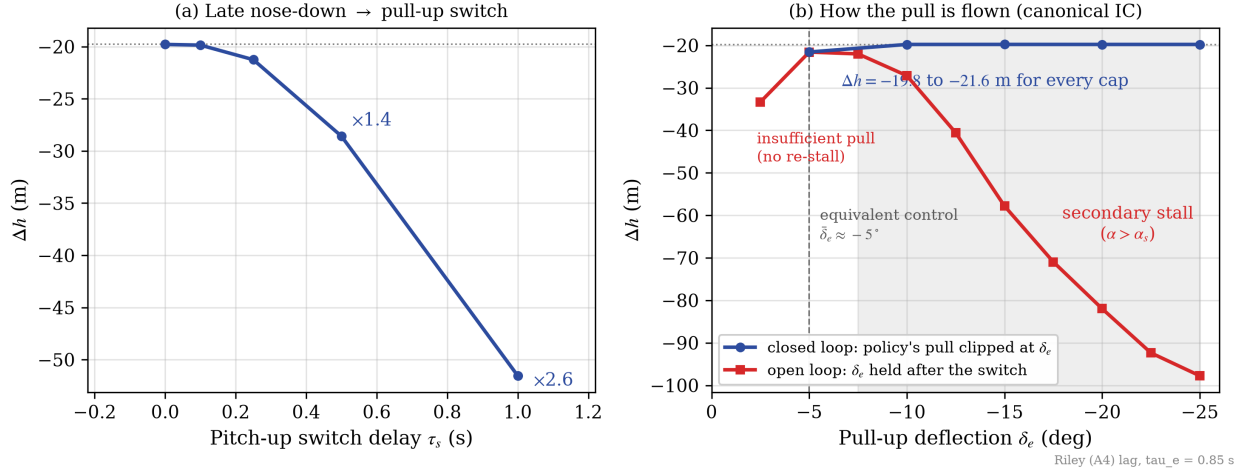


Fig. 11 Sensitivity of the recovery to pilot execution errors at the canonical IC. (a) Delaying the nose-down → pull-up switch by τ_s , with the loss multipliers annotated. (b) The pull phase flown in its two modes, on one signed deflection scale (axis inverted so that pulling harder reads rightward): closed loop (blue), the policy with its pull commands clipped at δ_e ; open loop (red), the same deflection held constant after the switch. Dotted line: the optimal loss; dashed vertical: the equivalent control $\bar{\delta}_e \approx -5^\circ$; shaded: the held pulls that re-stall the wing ($\alpha > \alpha_s$). Note that the abscissa means a different thing for each curve: the deflection actually flown for the open loop, only a ceiling for the closed loop.

Figure 11 shows that the two pitch axes could hardly be more asymmetric. The switch delay is the steepest axis of the entire study: at the canonical IC, holding the nose down 0.25 s too long costs 3.7 m, 0.5 s multiplies the loss by 3.0, and a single second multiplies it by 6.3 (−50 m); across the IC grid, one held second lands between −39 and −54 m. The mechanism compounds: every instant spent past the switch point is spent at full power in a steepening dive, so the delay converts to altitude superlinearly, unlike the power axis above, whose cost is a fixed ≈ 7.7 m per second.

The pull axis, by contrast, splits into two regimes that panel (b) overlays on the same deflection scale, and the contrast between them *is* the result. Flown closed loop, the pull is remarkably forgiving: capping the pull anywhere between -25° and -10° changes the loss by less than the ± 0.3 m execution band; the equivalent control that holds the sliding arc averages only $\approx -5^\circ$ (Sec. V.F), so any cap above it leaves enough authority, with the chattering simply saturating at the cap. The flatness of that curve is a direct reading of how little pull the policy actually asks for. Its demand averages -4.99° and exceeds 5° over just 5.7% of the pull phase, the large deflections being brief chattering spikes of the sliding mode that peak at -19.69° . A cap at -20° or beyond therefore never binds and reproduces the optimum exactly; caps of -15° and -10° clip only those spikes; and only at -5° , where the ceiling reaches the equivalent control itself, does the constraint bite. Nor does any clipped case re-enter the stall in a way the uncapped policy avoids: across every cap of panel (b) the angle of attack peaks at the same 14.4° the optimum itself reaches (Sec. V.F), marginally past α_s and identical across caps, because the clip bounds only how hard the pilot may pull, while the release that the regulation commands whenever α approaches the boundary passes through it untouched. The intermediate caps in fact *edge out* the uncapped execution, by up to 0.17 m. No optimality is violated: what is flown is the action blend, faithful to the Bellman valuation but not identical to it (Sec. V.D), and in the final approach to level flight its vertex mix

draws a spurious pull component from the terminal cells in which the recovery problem is not posed (Sec. V.E). Filling those cells from their nearest interior neighbour removes most of that component — the same correction that closed the throttle artifact above — and what survives is small: the mean pull command after the switch is -4.89° uncapped against the $\approx -5^\circ$ equivalent control. A cap trims the remainder (-4.82° when capped at 10°), the level-off closes 0.74 s sooner, and the gain stays inside the same ± 0.3 m band that bounds every execution-level effect in this paper. Flown *open loop*, the same deflections are catastrophic: the held-pull curve is a sharp, asymmetric valley whose minimum sits exactly at $\delta_e = -5^\circ$ (an independent confirmation of the equivalent-control value), and at and beyond -7.5° the constant pull drives the angle of attack back across α_s (secondary stall, shaded region: α_{max} grows monotonically from 17.0° to 35.4°) with losses escalating to -127 m at full deflection, the endpoint of the continuum whose scripted counterparts are the open-loop full-pull entries of Table 8 (-130 to -132 m at this entry, flown with their own power schedules rather than the policy’s). The valley’s two flanks fail by opposite mechanisms: -5° held merely grazes the boundary ($\alpha_{max} = 14.1^\circ$), the equivalent control held by hand, whereas -2.5° nearly triples the optimal loss (-20.97 m) with no re-stall at all, through insufficient lift and a level-off that drags on. Re-stall is therefore confined to a single branch and a single direction: deflection beyond -7.5° , *sustained*. The identical deflections that are harmless as a closed-loop cap re-stall the wing when held: what matters is not how hard the pull is, but whether it is flown on the angle of attack.

The operational hierarchy of the three error axes is then: the *promptness* of the pull, once the wing unstalls, dominates everything (superlinear); power timing comes second (linear, ≈ 7.7 m/s); and the pull *vigor* barely matters in closed loop while being catastrophic in open loop. For training emphasis the implication is direct: initiate the pull without hesitation and fly it on the stall boundary; the exact deflection used to get there is almost immaterial. This hierarchy also puts the doctrinal question of Sec. V.E in its operational place: the entire CAA-over-FAA margin (median 1.3 m over the certified domain) is smaller than the cost of a quarter-second of hesitation on the pull. The authorities’ disagreement is real and its resolution is now certified. But what recovery training should emphasize is not which sequence to fly, it is how promptly and how attentively the pull is flown.

I. Sensitivity to the Thrust Calibration

One model parameter is not measured but assumed, and it deserves its own accounting because it is the most consequential of the study. The Riley tables supply the aerodynamics; they do not supply the map from throttle setting to thrust. As stated in Sec. V.B, K_t is calibrated by requiring that full throttle sustain level cruise at $V_{max} = 2V_s$, giving $K_t \approx 1151$ N. Measured against the 108 hp continuous rating that Riley does document (Table 9), that calibration implies a propulsive efficiency of $\eta \approx 0.90$ at V_{max} . The propeller of this airframe is a two-blade fixed-pitch unit turning at 2600 rpm (Table 9), which at V_{max} places it at an advance ratio $J = V/(nD) \approx 0.81$. Full-scale tests of two-blade fixed-pitch propellers report, at that advance ratio, an efficiency near 0.76 at a representative fixed blade angle, rising to ≈ 0.85 only along the envelope obtained by *re-pitching* the blade for each condition — an option a fixed-pitch installation does not have — with an absolute peak of ≈ 0.87 reached at $J \approx 1.25$ – 1.55 [24]. The calibration therefore assumes a propulsive efficiency above the peak that propeller class attains anywhere, and roughly 18% above what a fixed blade angle delivers at the relevant operating point.

The consequence is not marginal. Flying the *nominal* policy on plants whose engine delivers $\eta \in [0.75, 0.95]$ of the modelled thrust — the pilot commands the same throttle fraction, the airframe receives less — moves the altitude loss at the canonical entry from 6.1 to 15.8 m, a factor of 2.6. For scale, the two model corrections that this section’s aerodynamics required move the same number by 0.4 and 0.9 m. The thrust calibration dominates every other uncertainty in the model by an order of magnitude, and no choice of η can be defended from the wind-tunnel data alone.

What survives the uncertainty is the comparison the paper is actually about. Across the same range, the ordering of the recovery strategies never changes,

$$\text{DP optimum} > \text{CAA} > \text{FAA} > \text{power-delayed},$$

and the margin of the optimum over the CAA doctrine stays between 7.9 and 9.2 m while the absolute losses swing by 9.7 m. The advantage of optimal control over procedural flying is therefore a property of the control law, not an artifact of how the propeller was calibrated; the absolute altitude figures should be read as conditional on $\eta \approx 0.90$, and rescaled

accordingly for an airframe whose propulsive efficiency is known.

VI. Conclusions

Exact dynamic programming for aircraft upset recovery has been memory-bound for as long as it has existed; this work makes it compute-bound. Integrating the transitions inside GPU registers instead of storing them reduces the footprint from $O(N_s N_a 2^D)$ to $O(N_s)$ and brings grids of 10^7 – 10^8 states within reach of a single GPU, while the non-expansive barycentric interpolation keeps the discretized backup consistent, so that what converges is the exact optimum of the discretized process rather than a sampled approximation of it. The instrument is validated on three independent lines: it reproduces the published value function and switching surfaces of the 3-DOF banked-pullout benchmark [2] in 0.38 s end to end, converges a 32-million-state refinement of the same problem (six hundred times the original grid) in under fifteen minutes, and agrees at trajectory level with direct collocation wherever the two formulations overlap.

Its first substantive application settles a question that certified guidance leaves open. On a 4-DOF symmetric stall model built from Riley’s propulsion-coupled wind-tunnel tables [3], the recovery sequence taught by the CAA *emerges* from the Bellman recursion as a property of the optimum: nothing in the formulation names a procedure, yet the converged policy commands full power essentially everywhere and switches from nose-down to pull on a surface pinned to the stall angle. The resulting ordering, optimum $\leq$ CAA $\leq$ FAA $\leq$ power-delayed, is certified per entry rather than on average: it holds without a single exception over the 2300 valid nodes of the entry planes and over 1000 Latin-hypercube samples of the full four-dimensional stalled-entry set, with a median CAA advantage of 1.32 m and a narrowest of 0.05 m.

The exact solution also separates what flight test cannot. Of the altitude either certified sequence concedes to the optimum, most is the engine: reaching full power through a 2 s ramp costs ≈ 4 –8 m at every entry, a floor no procedure on this aircraft can avoid, while the doctrine itself costs a median of 1.3 m and at most 4.2 m, largest exactly where altitude margin is scarcest; a deliberate 2 s pause dwarfs both at 13–23 m. Under matched power-ramp modeling the simulated CAA-to-power-delayed ratio at the natural stall break is 0.61–0.62, in close agreement with the ≈ 0.6 implied by the flight-test campaign of Gratton et al. [4]

The optimum is moreover flyable, because its structure is expressible in the language pilots already use. The nose-down-to-pull transition is a state event (the crossing of the stall boundary), not an instant on a clock, and the pull that follows rides just below α_s on an equivalent control of only $\approx -5^\circ$: full power at once, push until the wing unstalls, then pull to the edge of the usable lift and hold it there.

The sensitivity study then relocates the operational emphasis. The promptness of that pull dominates everything, converting hesitation to altitude superlinearly at 24–42 m per held second; power timing is linear at ≈ 7.7 m per second; and the vigor of the pull barely matters while it is flown on the angle of attack, yet becomes catastrophic the moment it is not. The entire doctrinal margin the authorities disagree over is worth less altitude than a quarter-second of hesitation on the pull.

The nominal policy is also robust to everything an operator cannot control: flown unchanged across the aircraft’s achievable loading envelope it never departs controlled flight, the switching surface survives the entire longitudinal CG range, and the heavy-overload failures are informational rather than aerodynamic: completing the level-off requires only that the stall speed used for normalization reflect the weight of the day, a single pre-flight scalar.

These claims are deliberately scoped: one airframe, one aerodynamic model, and the symmetric regime in which the certified procedures disagree and the flight tests were flown. The asymmetric departure and the developed spin require the lateral-directional states, and the compute-bound formulation is sized to reach the eight-state model they demand. The question that opened this paper, whether the recovery procedures aviation teaches are optimal or merely adequate, was unanswerable for want of a reference. For the symmetric stall of this aircraft, it is now answered; for the spin, the reference is within reach.

A. Riley (1985) Aircraft and Aerodynamic Data

This appendix collects the physical parameters of the Grumman American AA-1 Yankee (Table 9) and the complete transcription of the seven aerodynamic coefficient tables from Table III of Riley [3] (NASA TM-86309) used by the 4-DOF solver of Case II, at the two tabulated propulsive conditions ($C_T = 0$ power-off, $C_T = 0.5$ power-on). Elevator derivatives are converted from per-degree to rad^{-1} . These are the exact values embedded in the CUDA kernel.

Table 9 Physical parameters of the Grumman American AA-1 Yankee used in the 4-DOF model. Airframe values follow Table I of Riley [3], converted to SI units; derived quantities follow from the stated definitions.

Parameter	Symbol	Value	Units
<i>Airframe (Riley [3], Table I)</i>			
Mass	m	715.21	kg
Pitch moment of inertia	I_{yy}	1000.60	kg m^2
Wing reference area	S	9.1147	m^2
Mean aerodynamic chord	$\bar{c}$	1.22	m
Moment reference (CG) position	x_{cg}	0.25	$\bar{c}$
<i>Environment and aerodynamic reference</i>			
Air density (sea level)	ρ	1.225	kg/m^3
Gravitational acceleration	g	9.81	m/s^2
Maximum power-off lift coefficient	$C_{L_{stall}}$	1.26	—
Stall angle of attack	α_s	14	deg
<i>Derived</i>			
Stall speed, $V_s = \sqrt{2mg/(\rho S C_{L_{stall}})}$	V_s	31.58	m/s
Calibration cruise speed, $2V_s$	V_{max}	63.16	m/s
Thrust gain, $T = K_t \delta_t$	K_t	1151	N

Table 10 Riley Table IIIa: C_{L_o} vs. α at $C_T = 0$ and $C_T = 0.5$.

α (deg)	C_{L_o} ($C_T=0$)	C_{L_o} ($C_T=0.5$)
−10	−0.41	−0.67
−5	−0.01	−0.14
0	0.41	0.41
5	0.84	0.97
10	1.16	1.42
12	1.23	1.54
14	1.26	1.62
16	1.26	1.67
18	1.26	1.72
20	1.25	1.76
25	1.22	1.85
30	1.17	1.92
35	1.13	1.99
40	1.08	2.05

Table 11 Riley Table IIIa: $C_{L_{\dot{q}}}$ vs. α at $C_T = 0$ and $C_T = 0.5$.

α (deg)	$C_{L_{\dot{q}}} (C_T=0)$	$C_{L_{\dot{q}}} (C_T=0.5)$
-10	2.41	3.01
-5	2.41	3.01
0	2.42	3.03
5	2.46	3.22
10	2.59	3.59
12	2.96	4.35
14	3.72	6.07
16	4.73	6.38
18	5.29	6.99
20	5.16	6.83
25	5.05	6.56
30	5.06	6.13
35	5.98	5.97
40	5.08	5.81

Table 12 Riley Table IIIa: $C_{L_{\delta_e}}$ vs. α at $C_T = 0$ and $C_T = 0.5$ (rad^{-1}).

α (deg)	$C_{L_{\delta_e}} (C_T=0)$	$C_{L_{\delta_e}} (C_T=0.5)$
-10	0.355	0.796
-5	0.361	0.779
0	0.355	0.750
5	0.332	0.705
10	0.304	0.624
12	0.292	0.595
14	0.286	0.578
16	0.281	0.561
18	0.275	0.538
20	0.269	0.515
25	0.252	0.458
30	0.241	0.418
35	0.223	0.349
40	0.212	0.315

Table 13 Riley Table IIIb: C_{D_o} vs. α at $C_T = 0$ and $C_T = 0.5$. The $C_T = 0.5$ column is the net axial-force coefficient of Eq. (14).

α (deg)	C_{D_o} ($C_T=0$)	C_{D_o} ($C_T=0.5$)
-10	0.0666	-0.3273
-5	0.0486	-0.3494
0	0.0526	-0.3474
5	0.0846	-0.3139
10	0.1456	-0.2483
12	0.1856	-0.2057
14	0.2446	-0.1435
16	0.3136	-0.0709
18	0.3786	-0.0018
20	0.4486	0.0727
25	0.6186	0.2561
30	0.7786	0.4322
35	0.9255	0.5979
40	1.0636	0.7572

Table 14 Riley Table IIIc: C_{m_o} vs. α at $C_T = 0$ and $C_T = 0.5$, about the reference CG ($0.25 \bar{c}$).

α (deg)	C_{m_o} ($C_T=0$)	C_{m_o} ($C_T=0.5$)
-10	0.270	0.270
-5	0.158	0.158
0	0.076	0.076
5	0.002	0.002
10	-0.080	-0.080
12	-0.118	-0.118
14	-0.167	-0.167
16	-0.225	-0.225
18	-0.277	-0.277
20	-0.316	-0.316
25	-0.408	-0.408
30	-0.480	-0.480
35	-0.556	-0.556
40	-0.606	-0.606

Table 15 Riley Table IIIc: $C_{m_{\dot{q}}}$ vs. α at $C_T = 0$ and $C_T = 0.5$.

α (deg)	$C_{m_{\dot{q}}} (C_T=0)$	$C_{m_{\dot{q}}} (C_T=0.5)$
-10	-7.00	-8.75
-5	-7.00	-8.75
0	-7.04	-8.80
5	-7.15	-9.36
10	-7.52	-10.44
12	-8.62	-12.64
14	-10.80	-17.64
16	-13.73	-18.54
18	-15.38	-20.30
20	-15.00	-19.85
25	-14.66	-19.06
30	-14.71	-17.80
35	-14.77	-17.33
40	-14.77	-16.88

Table 16 Riley Table IIIc: $C_{m_{\delta_e}}$ vs. α at $C_T = 0$ and $C_T = 0.5$ (rad^{-1}).

α (deg)	$C_{m_{\delta_e}} (C_T=0)$	$C_{m_{\delta_e}} (C_T=0.5)$
-10	-1.105	-2.142
-5	-1.105	-2.250
0	-1.105	-2.256
5	-1.031	-2.262
10	-0.945	-2.199
12	-0.939	-2.062
14	-0.933	-1.912
16	-0.928	-1.781
18	-0.928	-1.650
20	-0.928	-1.541
25	-0.928	-1.294
30	-0.859	-1.220
35	-0.745	-1.088
40	-0.573	-0.859

References

- [1] Munos, R., and Moore, A. W., "Barycentric Interpolators for Continuous Space and Time Reinforcement Learning," *Advances in Neural Information Processing Systems*, Vol. 11, 1998.
- [2] Bunge, R. A., and Kroo, I. M., "Automatic Spin Recovery with Minimal Altitude Loss," *2018 AIAA Guidance, Navigation, and Control Conference*, Kissimmee, FL, 2018, pp. 1–13. <https://doi.org/10.2514/6.2018-1866>.
- [3] Riley, D. R., "Simulator Study of the Stall Departure Characteristics of a Light General Aviation Airplane with and without a Wing-Leading-Edge Modification," Tech. Rep. NASA-TM-86309, NASA Langley Research Center, Hampton, VA, 1985.
- [4] Gratton, G. B., Hoff, R. I., Bromfield, M., Rahman, A., Harbour, S., and Williams, S., "Evaluating a Set of Stall Recovery Actions for Single Engine Light Aeroplanes," *The Aeronautical Journal*, Vol. 118, No. 1203, 2014, pp. 461–480. <https://doi.org/10.1017/S0001924000009313>.
- [5] Belcastro, C. M., and Foster, J. V., "Aircraft loss-of-control accident analysis," *AIAA Guidance, Navigation, and Control Conference*, 2010. <https://doi.org/10.2514/6.2010-8004>, aIAA Paper 2010-8004.

- [6] Federal Aviation Administration, “Airplane Flying Handbook, Chapter 5: Maintaining Aircraft Control: Upset Prevention and Recovery Training,” Tech. Rep. FAA-H-8083-3C, U.S. Department of Transportation, Federal Aviation Administration, Washington, DC, 2021. https://www.faa.gov/regulations_policies/handbooks_manuals/aviation/airplane_handbook.
- [7] UK Civil Aviation Authority, “Stall/Spin Awareness,” Handling Sense Leaflet 02, v2, Civil Aviation Authority, 2009.
- [8] Bellman, R., *Dynamic Programming*, Princeton University Press, Princeton, NJ, 1957.
- [9] Bertsekas, D. P., *Dynamic Programming and Optimal Control*, 4th ed., Vol. 1, Athena Scientific, Belmont, MA, 2017.
- [10] Crespo, L. G., Kenny, S. P., Cox, D. E., and Murri, D. G., “Analysis of control strategies for aircraft flight upset recovery,” *AIAA Guidance, Navigation, and Control Conference*, 2012. <https://doi.org/10.2514/6.2012-5026>, aIAA Paper 2012-5026.
- [11] Cunis, T., Burlion, L., and Condomines, J.-P., “Piecewise polynomial modeling for control and analysis of aircraft dynamics beyond stall,” *Journal of Guidance, Control, and Dynamics*, Vol. 42, No. 4, 2019, pp. 949–957. <https://doi.org/10.2514/1.6003618>.
- [12] Grillo, A., Torre, G. G., and Bunge, R. A., “Optimal Stall Recovery via Deep Reinforcement Learning for a General Aviation Aircraft,” *AIAA SCITECH 2024 Forum*, Orlando, FL, 2024. <https://doi.org/10.2514/6.2024-1280>.
- [13] Munos, R., and Moore, A. W., “Variable Resolution Discretization in Optimal Control,” *Machine Learning*, Vol. 49, No. 2–3, 2002, pp. 291–323. <https://doi.org/10.1023/A:1017992615625>.
- [14] Federal Aviation Administration, “Stall and Spin Awareness Training,” Tech. Rep. Advisory Circular AC 61-67C, U.S. Department of Transportation, Federal Aviation Administration, Washington, DC, 2000. With Changes 1–2.
- [15] Stough, I., H. Paul, DiCarlo, D. J., and Patton, J., James M., “Flight investigation of stall, spin, and recovery characteristics of a low-wing, single-engine, T-tail light airplane,” Tech. Rep. NASA TP-2427, NASA Langley Research Center, 1985.
- [16] Bui, M., Hu, H., He, C., Lu, M., Giovanis, G., Shriraman, A., and Chen, M., “OptimizedDP: an efficient, user-friendly library for optimal control and dynamic programming,” *arXiv preprint arXiv:2204.05520*, 2022.
- [17] Courant, R., Friedrichs, K., and Lewy, H., “On the Partial Difference Equations of Mathematical Physics,” *IBM Journal of Research and Development*, Vol. 11, No. 2, 1967, pp. 215–234. <https://doi.org/10.1147/rd.112.0215>, english translation of the 1928 original in *Mathematische Annalen*.
- [18] Betts, J. T., “Survey of numerical methods for trajectory optimization,” *Journal of Guidance, Control, and Dynamics*, Vol. 21, No. 2, 1998, pp. 193–207. <https://doi.org/10.2514/2.4231>.
- [19] Andersson, J. A. E., Gillis, J., Horn, G., Rawlings, J. B., and Diehl, M., “CasADi: a software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, Vol. 11, No. 1, 2019, pp. 1–36. <https://doi.org/10.1007/s12532-018-0139-4>.
- [20] Wächter, A., and Biegler, L. T., “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, Vol. 106, No. 1, 2006, pp. 25–57. <https://doi.org/10.1007/s10107-004-0559-y>.
- [21] Filippov, A. F., *Differential Equations with Discontinuous Righthand Sides*, Mathematics and Its Applications (Soviet Series), Vol. 18, Kluwer Academic Publishers, Dordrecht, 1988. <https://doi.org/10.1007/978-94-015-7793-9>.
- [22] Utkin, V. I., “Variable structure systems with sliding modes,” *IEEE Transactions on Automatic Control*, Vol. AC-22, No. 2, 1977, pp. 212–222. <https://doi.org/10.1109/TAC.1977.1101446>.
- [23] Bryson, A. E., and Ho, Y.-C., *Applied Optimal Control: Optimization, Estimation, and Control*, Hemisphere Publishing, Washington, DC, 1975.
- [24] Hartman, E. P., and Biermann, D., “The Aerodynamic Characteristics of Full-Scale Propellers Having 2, 3, and 4 Blades of Clark Y and R.A.F. 6 Airfoil Sections,” Tech. Rep. NACA-TR-640, National Advisory Committee for Aeronautics, Washington, DC, 1938. URL <https://ntrs.nasa.gov/citations/19930091715>, aerodynamic tests of seven full-scale 10-ft-diameter propellers.